DRIVING EMBEDDED EXCELLENCE

Notes on Debugging VRTA Applications

Table of Contents

1 Introduction 3
2 Considerations when debugging VRTA 4
2.1 Threads 4
2.2 Interrupts (Windows) 4
2.3 Interrupts (Linux) 4
2.4 Source code location and for Eve Projects 4
3 Debugging with gdb (MinGW) 6
4 Debugging with gdb (RTPC) 7
5 Debugging with VisualStudio 8
5.1 VS2010 8
5.2 Debugging with VisualStudio (VS2012) 9
6 Debugging with Bake 10
7 Debugging with Isolar-EVE (MinGW) 11
7.1 Attaching to an application in Debug Mode 12
8 Remote Debugging with Isolar-EVE (RTPC) 14
8.1 Setting up a Remote Debugging Configuration 18
9 Debugging with duplicate source files using Isolar-EVE 20
10 Debugging with Eclipse (MinGW) 22
10.1 Debug application 22
10.2 Debug by Attaching to an application 24
11 Remote Debugging with Eclipse (RTPC) 26
11.1 Prerequisites 26
11.2 Setting up Eclipse for Remote Debugging 28
11.3 Setting up a Remote Debugging Configuration 33
12 Debugging with duplicate source files using Eclipse 34
13 Debugging with NetBeans (MinGW) 36
14 NetBeans additional – Building and debugging Eve projects 38

1 Introduction

VRTA applications can be debugged in the same way as any normal C/C++ based program. They are simply 32-bit programs that run under Windows or Linux.

This document gives some tips about how to set up debugging sessions for a variety of different debuggers, along with a short discussion about special issues that need to be considered when debugging VECUs.

2 Considerations when debugging VRTA

2.1 Threads

VRTA applications use several threads. The first (*main* or *application*) thread is used to start and monitor the application. Other threads deal with timers and communication with external programs. However the thread that you are most likely to want to debug is the one that is running the OS application (often the third thread). The easiest way to find this thread is to set a break point on `vrta_app_main` (the `OS_MAIN` macro evaluates to `vrta_app_main`) and run to that when you start the debugging session.

2.2 Interrupts (Windows)

External interrupts (such as the SystemClock timer interrupt) are injected into VRTA-win ECUs from a separate thread outside the OS thread. This thread freezes the OS thread and modifies its stack to inject the interrupt handling code on top of the code that was originally running. When the thread is unfrozen, it executes the interrupt code and then resumes running the code that was preempted.

The Windows APIs that VRTA uses to inject these interrupts are the same APIs that debuggers use when they take control of programs, which can cause conflicts and leave the debugger in a confused state. Code has been added to VRTA to try to minimize these effects, but it cannot be 100% effective without some feedback from the debugger.

A global variable '`VRTA_DEBUG_DEFER_INTERRUPTS`' exists in every VRTA-win executable. Its value is normally zero. If it gets set to a non-zero value, then VRTA will postpone any external interrupts until it gets set back to zero.

A debugger that is single-stepping should set `VRTA_DEBUG_DEFER_INTERRUPTS = 1` to avoid interrupts happening during stepping. It must set it back to zero before it does a continue/run operation.

Firstly this helps to avoid confusing the debugger, and secondly it ensures that when you are single-stepping code in the OS thread, you don't suddenly jump into the interrupt handling code.

We supply helper code for gdb and VisualStudio based debuggers to make this happen automatically (described later).

2.3 Interrupts (Linux)

On a Linux system, interrupts are injected into the OS thread using Linux real-time signals. They are handled in the OS thread and on its stack, so they behave just the same as interrupts on a real ECU.

The gdb helper code that we supply sets up the debugger so that interrupts do not get injected while you are single-stepping in the OS thread.

2.4 Source code location and for Eve Projects

The EVE build process is designed to make builds as fast as possible. Compilation speed is increased by ensuring that all of the source files get built in a single folder (VECUs\<target>\Sources). The files that are in VECUs\<target>\Sources are **copies** of the originals. It is these files that the debugger 'sees' as the

reference source files. If you edit these files when inside the debugger, then you are editing the copies rather than the real master sources. Subsequent builds will not normally take account of your changes.

This behavior might change in later versions of the Eve build process.

3 Debugging with gdb (MinGW)

The command-line version of `gdb` can be used to debug VRTA-win ECUs that are built using the MinGW toolchain.

For gdb-based debugging, we provide a file called `vrta_support_for_gdb` that contains code that hooks certain debugger events that know how to set `VRTA_DEBUG_DEFER_INTERRUPTS` correctly. You should add a source reference to this file in your `.gdbinit` file before trying to debug VECUs. The hooks are able to detect non-VRTA executables and will not have any effect upon them, so the source line can be added to your default `.gdbinit` file without causing problems when debugging non-VRTA applications.

If you run `gdb.exe` from the command-line you need to ensure that the `vrta_support_for_gdb` file gets loaded. An example command line is:

```
gdb.exe -nx --ix="c:\etas\rta-os\bin\vrta_support_for_gdb" ecu.exe
```

Alternatively you can use the `gdb.bat` file that ships with VRTA. This inserts the necessary command-line options for you. Then you can just use:

```
gdb.bat ecu.exe
```

(If `gdb.bat` is on your path before `gdb.exe`, then you can just type '`gdb ecu.exe`')

See also below - Debugging with Bake.

4 Debugging with gdb (RTPC)

The command-line version of `gdb` on a Windows PC can be used to remote-debug VRTA-ux ECUs that are compiled on an RTPC (Linux PC).

You need to run `gdbserver` on the Linux PC and specify a port to listen on (e.g. `gdbserver :1000 ecu.vrta_ecu`).

Then on the Windows PC create a file `remote_gdb_info` with content based on:

```
directory ../Source
symbol-file ecu.vrta_ecu
target remote 192.168.1.14:1000
```

Then run:

```
gdb.exe -nx --ix="c:\etas\rta-os\bin\vrta_support_for_gdb" --ix=”loadgdb”
--ix=” remote_gdb_info”
```

Alternatively you can use the `gdb.bat` file that ships with VRTA. This inserts the necessary command-line options for you:

```
gdb.bat "-ix=remote_gdb_info"
```

See also below - Debugging with Bake.

5 Debugging with VisualStudio

See also below - Debugging with Bake.

5.1 VS2010

VRTA ships with macros for Visual Studio 2010 that automate the manipulation of the `VRTA_DEBUG_DEFER_INTERRUPTS` variable. They do not interfere with debugging of non-VRTA programs.

To load the macros:

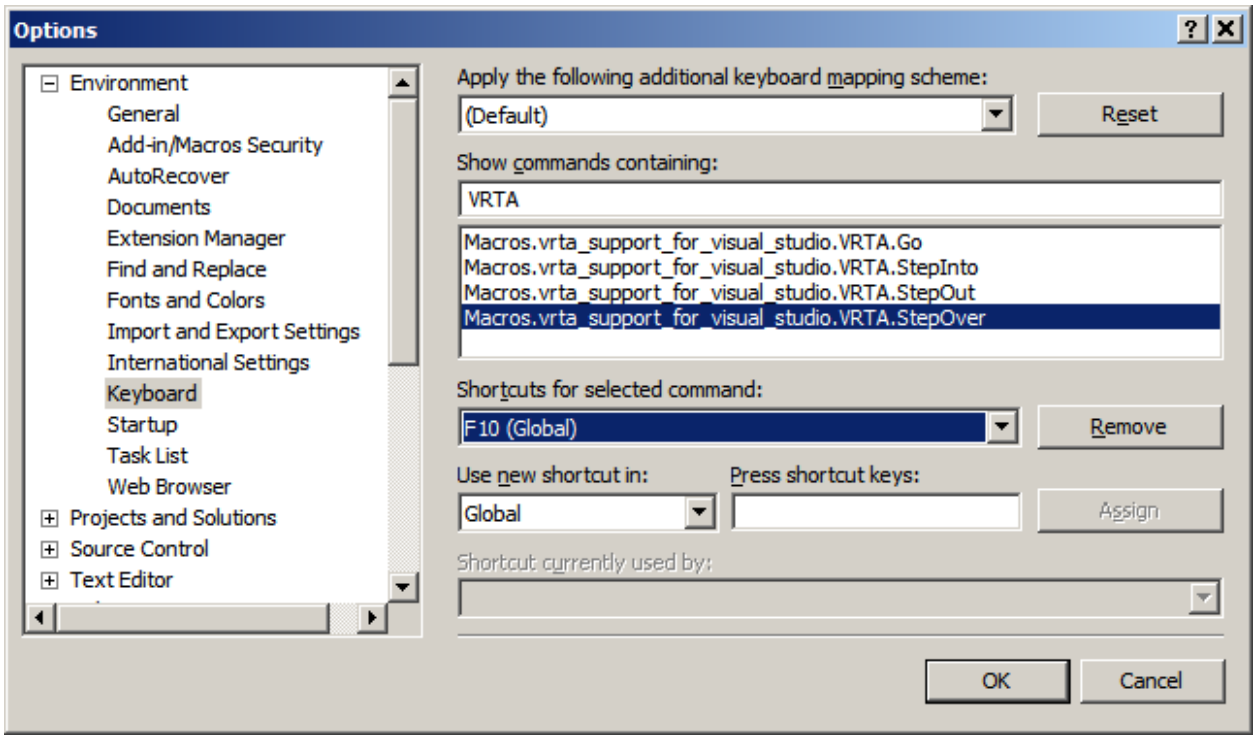
- Select Tools →Macros → Load Macro Project…
- Load 'vrta_support_for_visual_studio.vsmacros' from the VRTA 'bin' directory

The macros are named Go, StepInto, StepOut and StepOver. They are in the `VRTA` module.

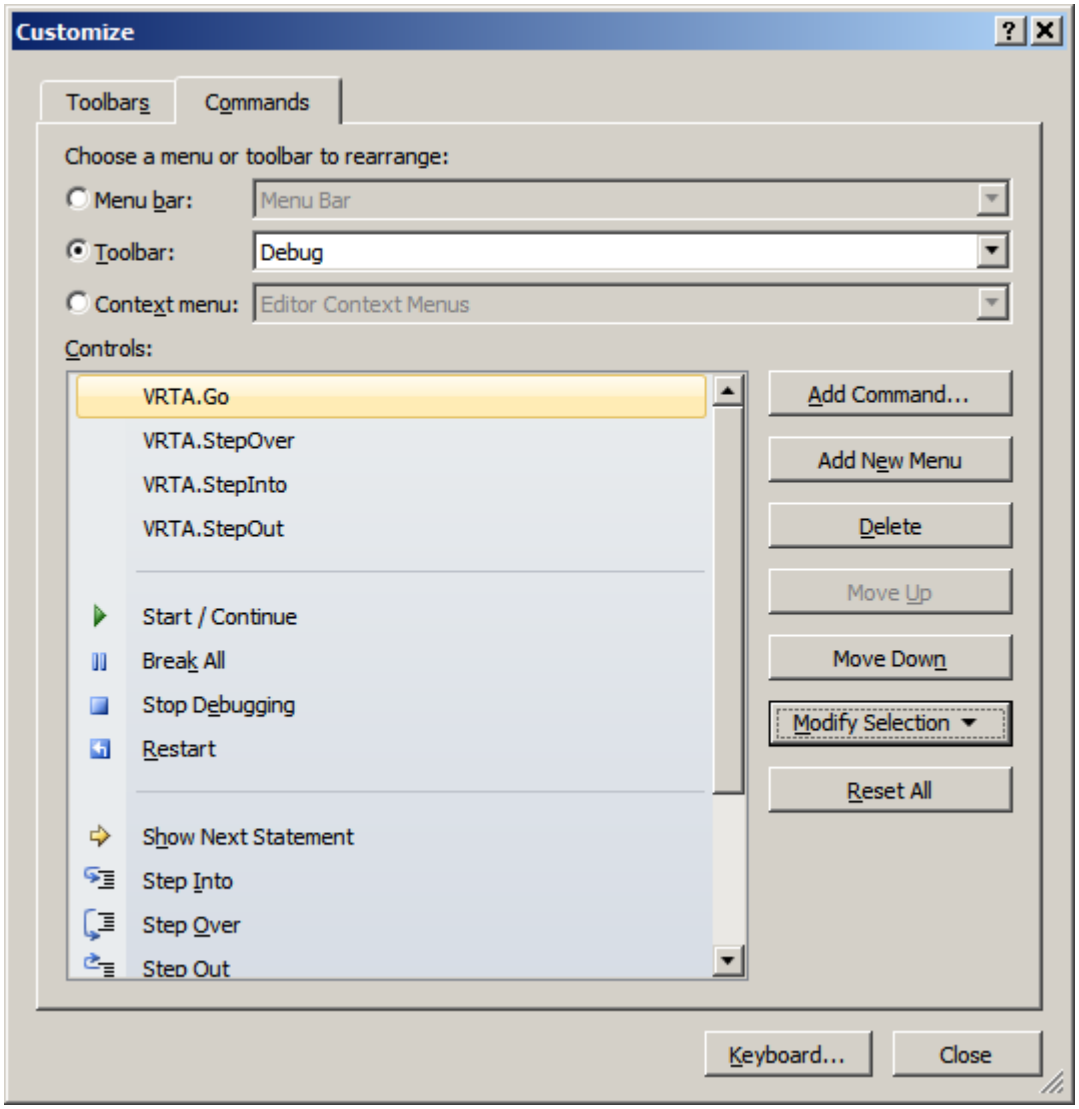
You can use the Macro Explorer to run them (double –click).

If you want to re-map the normal debugging keys to use these macros,

- Select Tools → Options → Environment → Keyboard. Select 'Show commands containing: VRTA)
- Map the 4 macros to F5, F11, Shift-F11 and F10 (or your own choice)



It is also possible (though tricky) to add these macros to the Debug toolbar:



5.2 Debugging with VisualStudio (VS2012)

Visual Studio 2012 has removed support for macros. At the moment we do not have an automated solution for manipulating `VRTA_DEBUG_DEFER_INTERRUPTS` in Visual Studio 2012.

6 Debugging with Bake

When you build a project using the Bake tool (for example when using Isolar-EVE), you can simply run ‘`bake debug`’ to start a debugging session.

Bake knows what type of target you are building, so will launch the appropriate debugger depending on the target type:

- **MinGW:** Starts a command-line gdb session. It automatically loads the `vrta_support_for_gdb` file and runs to OS_MAIN().

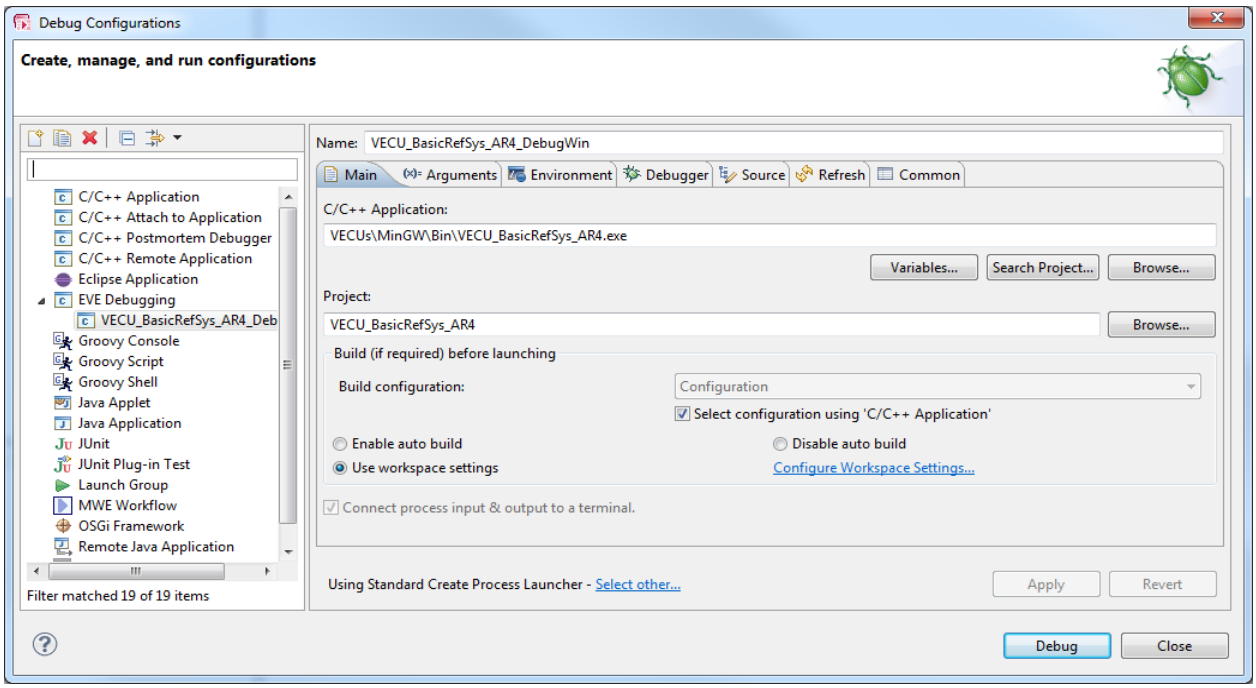
- **RTPC:** Starts a remote command-line gdb session. It automatically loads the `vrta_support_for_gdb` file and runs to OS_MAIN().

- **VisualStudio:** Starts Visual Studio and opens the file containing OS_MAIN().

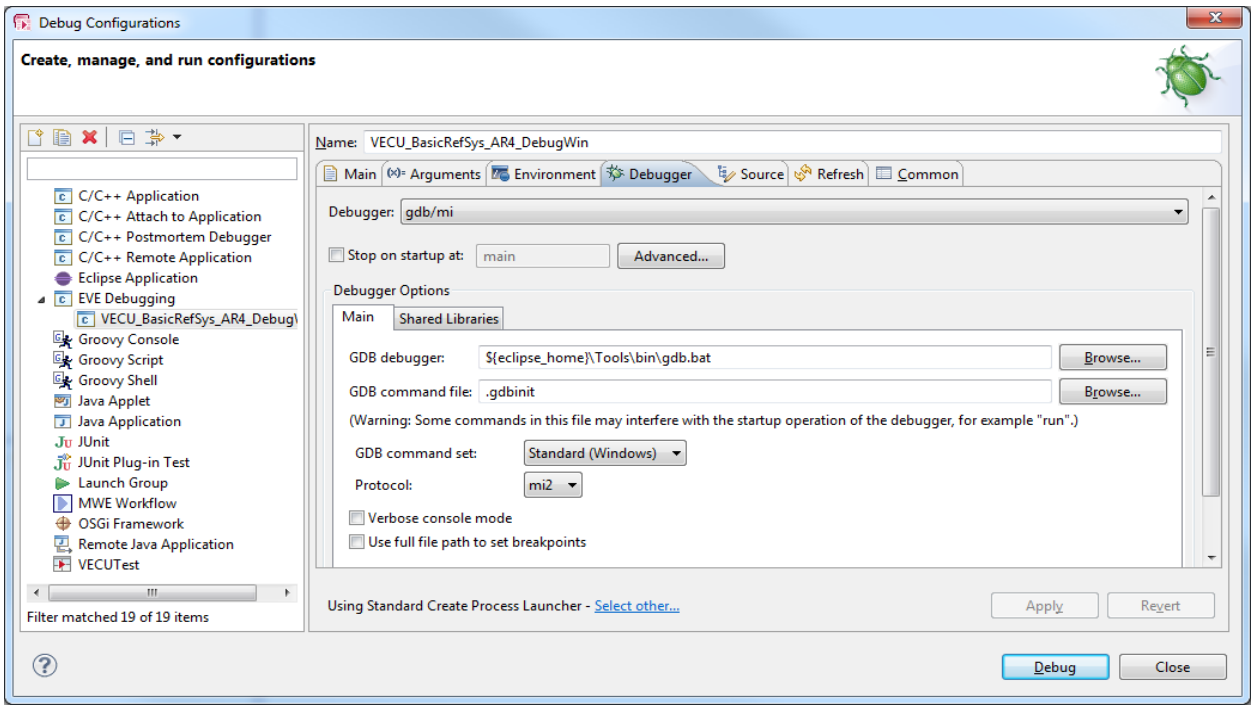
7 Debugging with Isolar-EVE (MinGW)

VRTA-win VECUs that are built by Isolar-EVE can be debugged using Eve's GUI front-end to gdb.

Right click project **Debug** As **-> Debug Configurations.** Under **C/C++ Application**, select **'EVE Debugging'**.



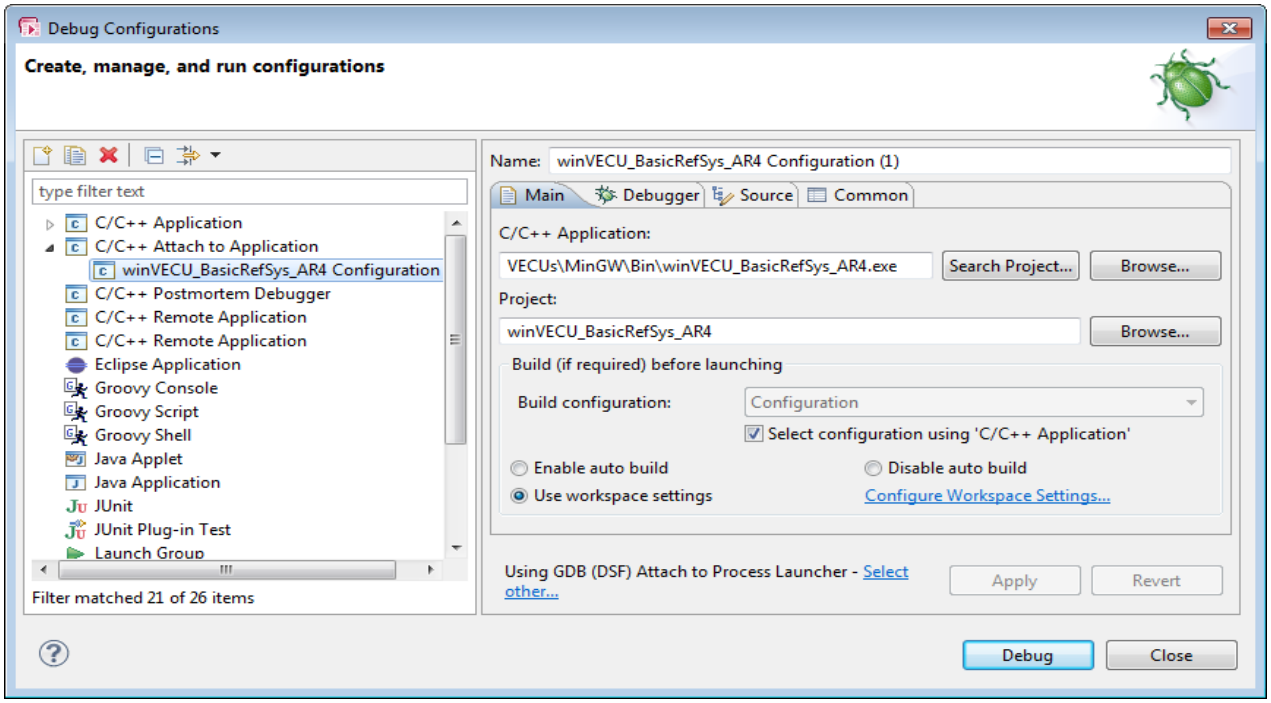
Debugger Settings



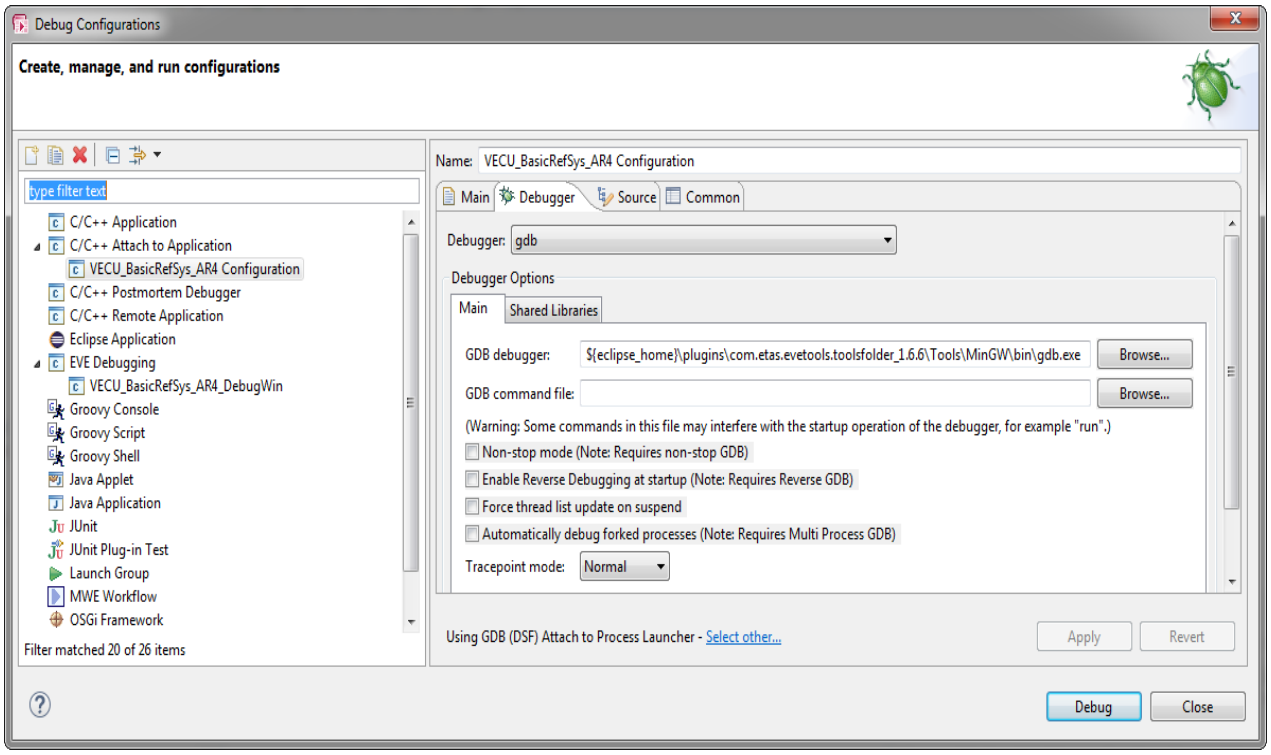
7.1 Attaching to an application in Debug Mode

You can also attach to an already running VRTA-win VECU.

Select **Run -> Debug Configurations.** Under **C/C++ Attach to Application**, select your launch configuration.



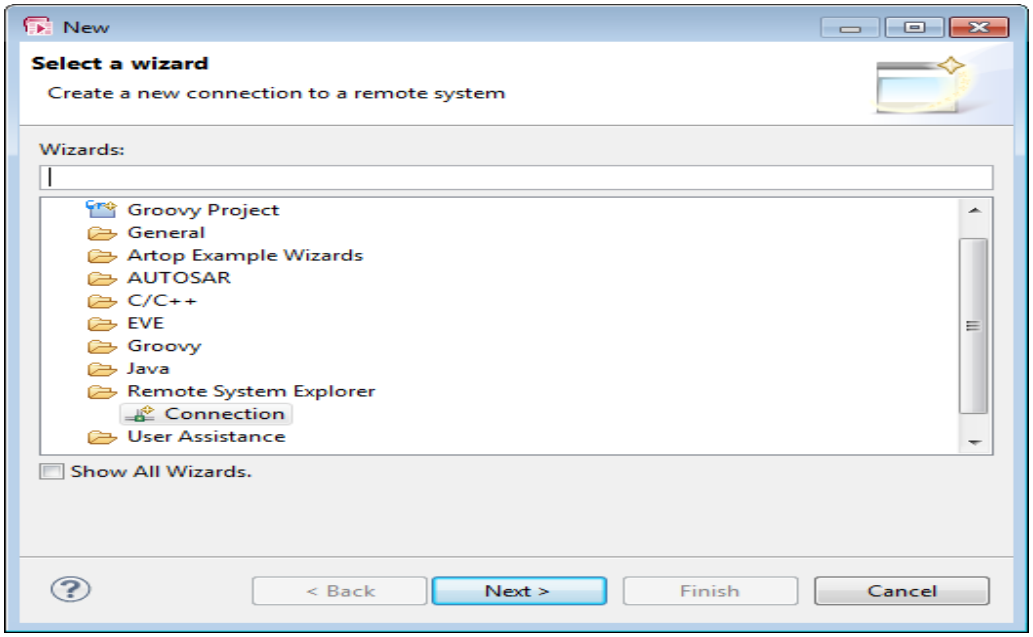
Debugger Settings



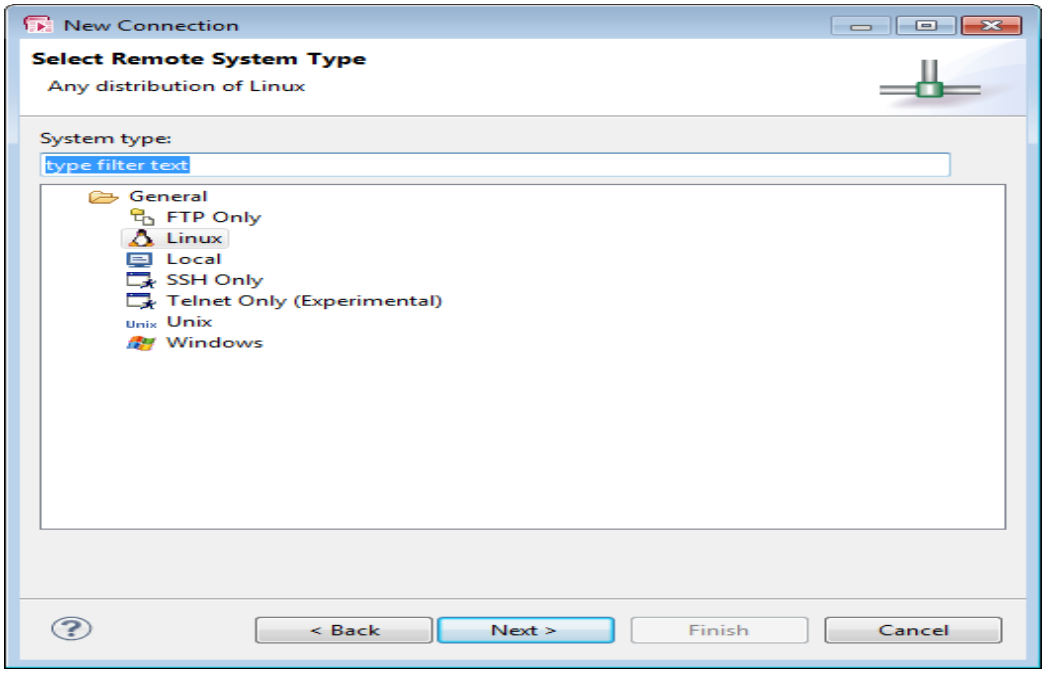
8 Remote Debugging with Isolar-EVE (RTPC)

VRTA-ux VECUs that are built by Isolar-EVE can be debugged remotely using Eve's GUI front-end to gdb.

Click **File -> New -> Other**. In the window that appears, double-click **Remote System Explorer** to expand it and select **Connection**.

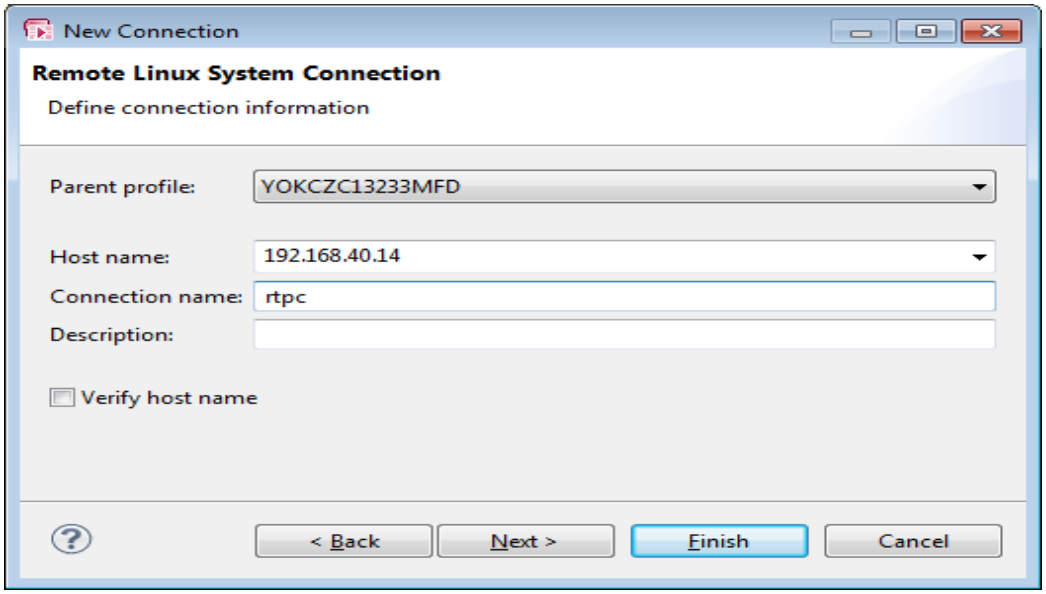


Click **Next**. Select **Linux**

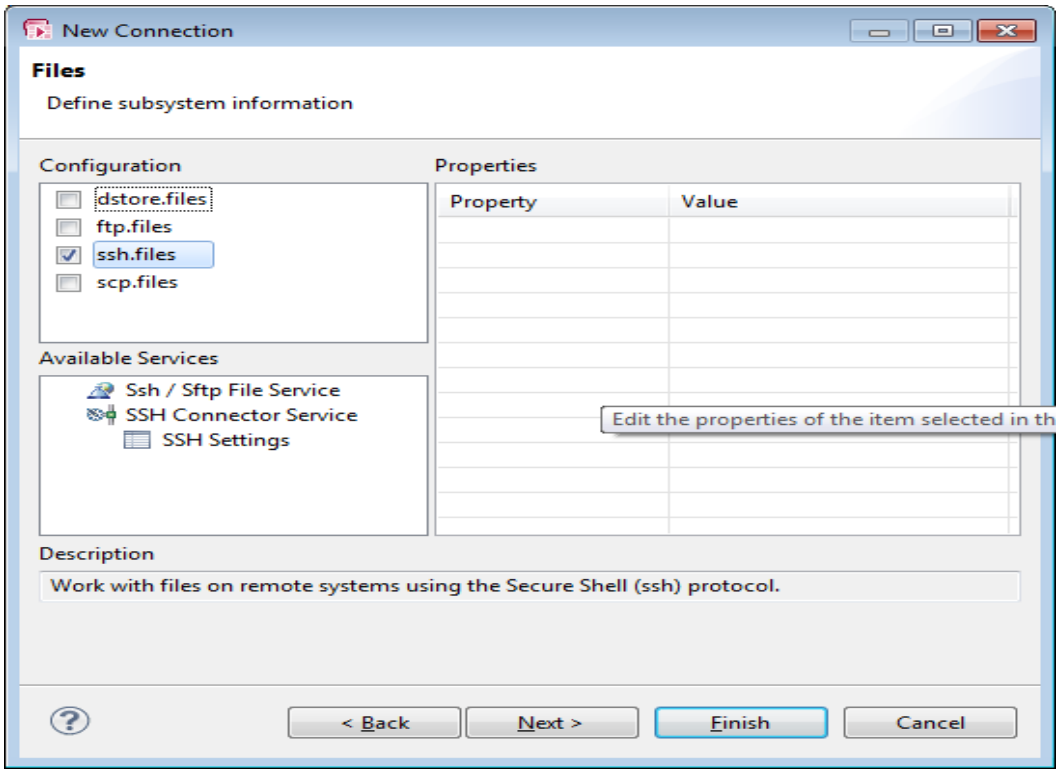


Click **Next**. At **Host name**, enter the IP address of the rtpc (192.168.40.14 in the example).

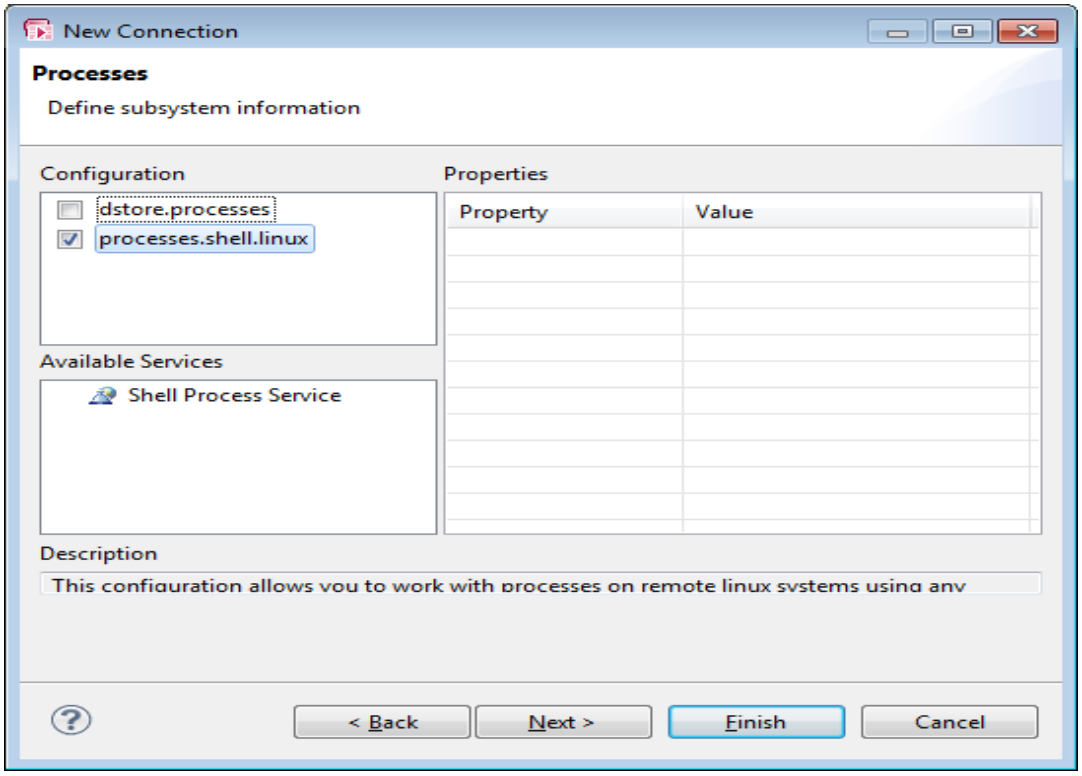
At **Connection name**, enter a name for the connection (rtpc in the example).



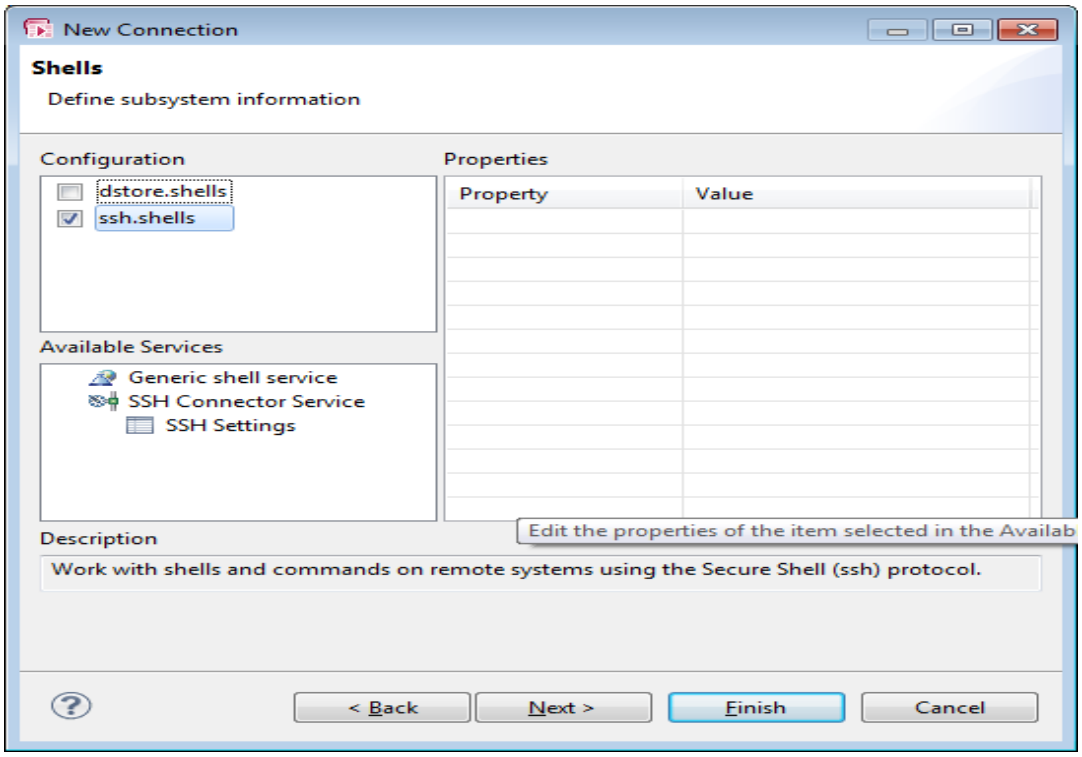
Click **Next**. Under **Configuration**, select **ssh.files**.



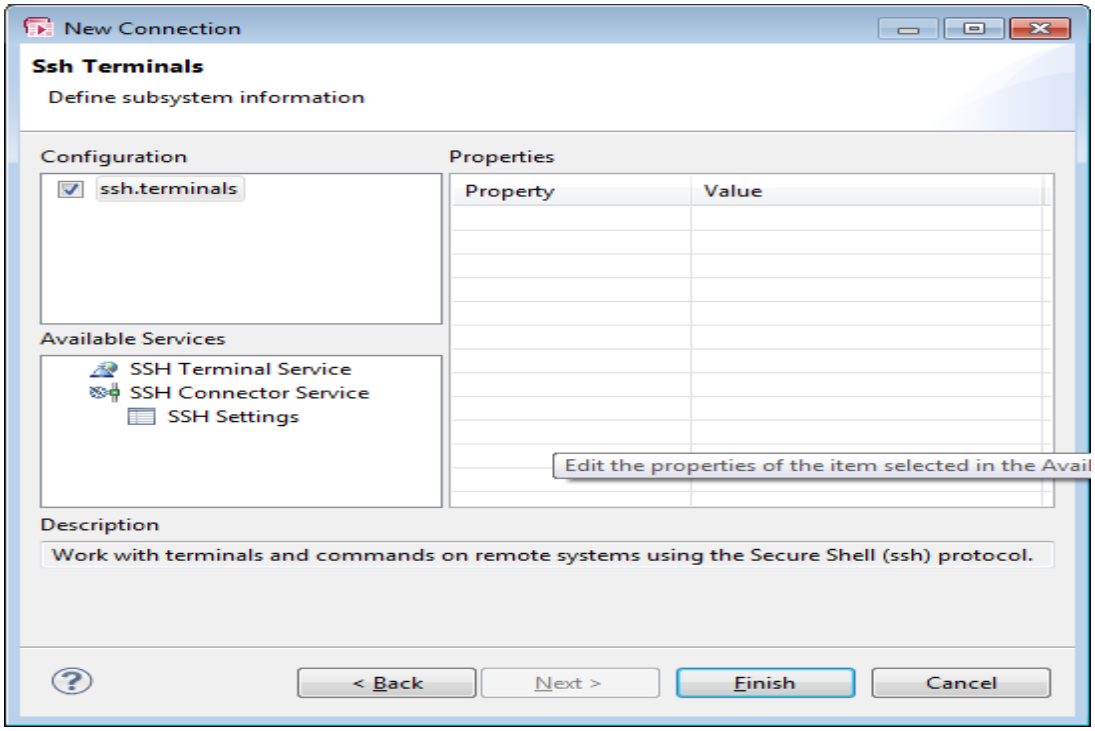
Click **Next**. Select **processes.shell.Linux**



Click **Next**. Select **ssh.shells**.

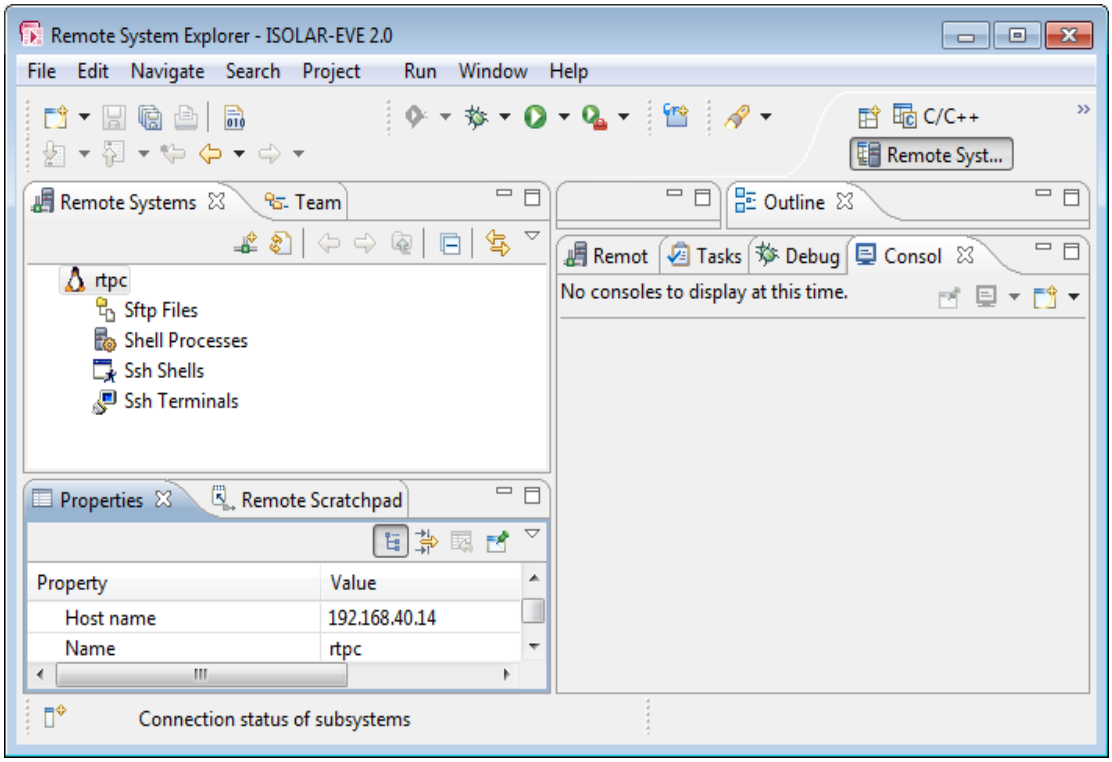


Click **Next**. Select **ssh.terminals**.

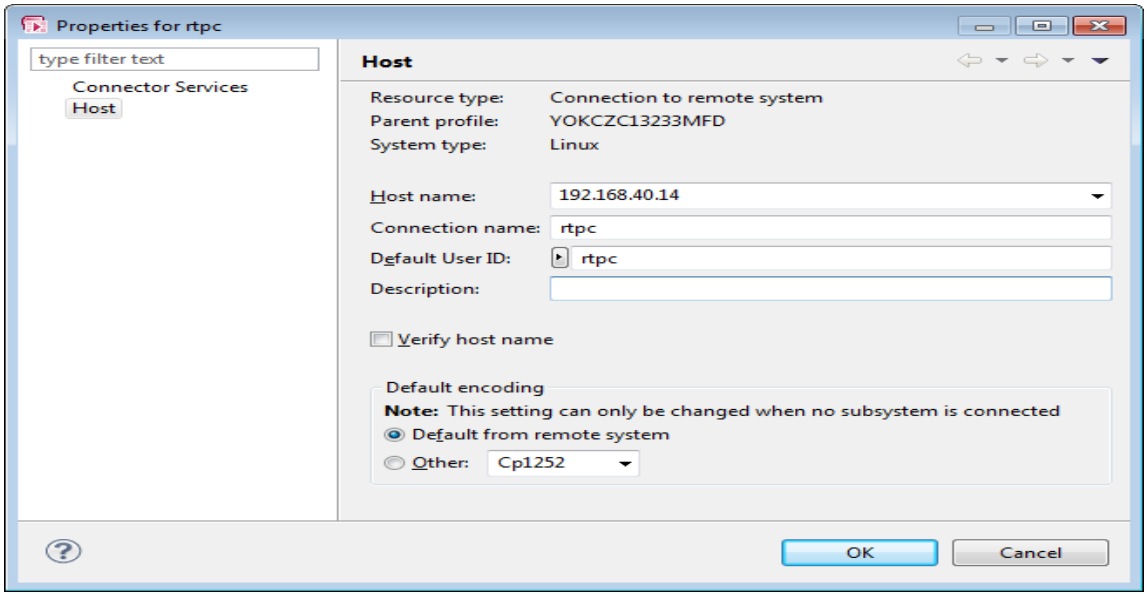


Click **Finish**.

In the **Remote Systems** tab, right-click the connection and select **Properties**.



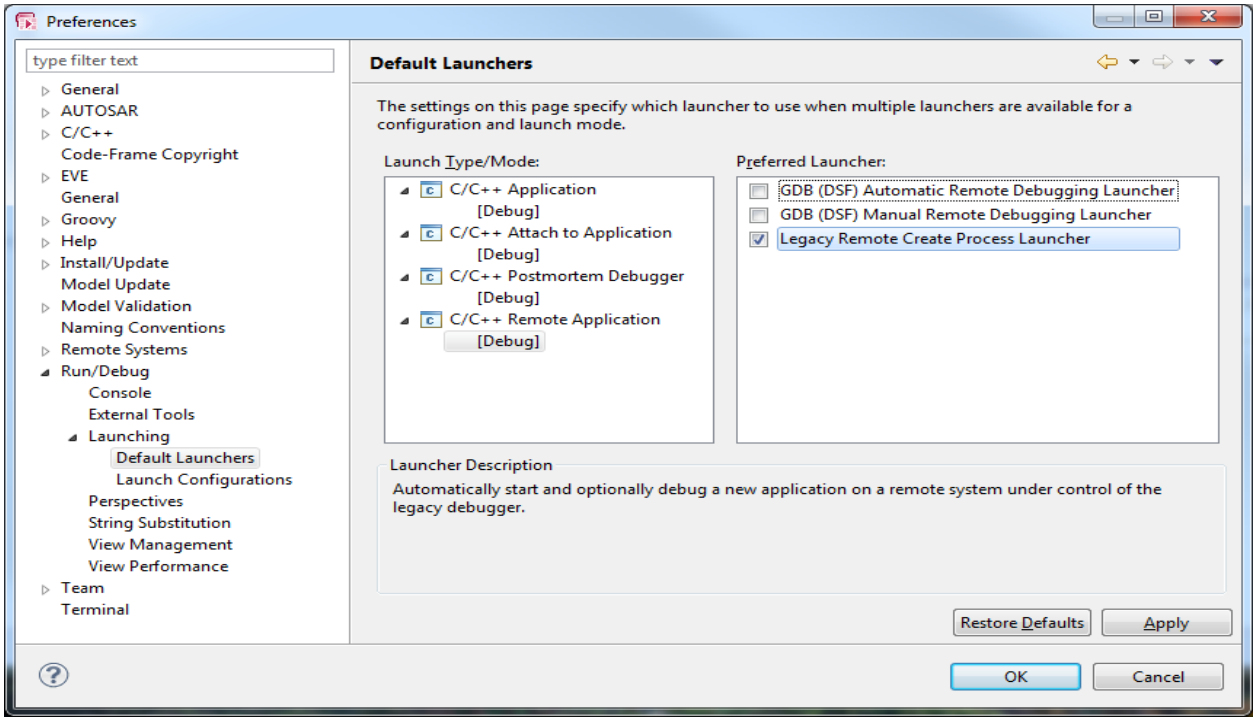
Set the connection's user to **rtpc**: click **Host**. Click the small icon just to the right of **Default User ID** and enter **rtpc and click OK.**



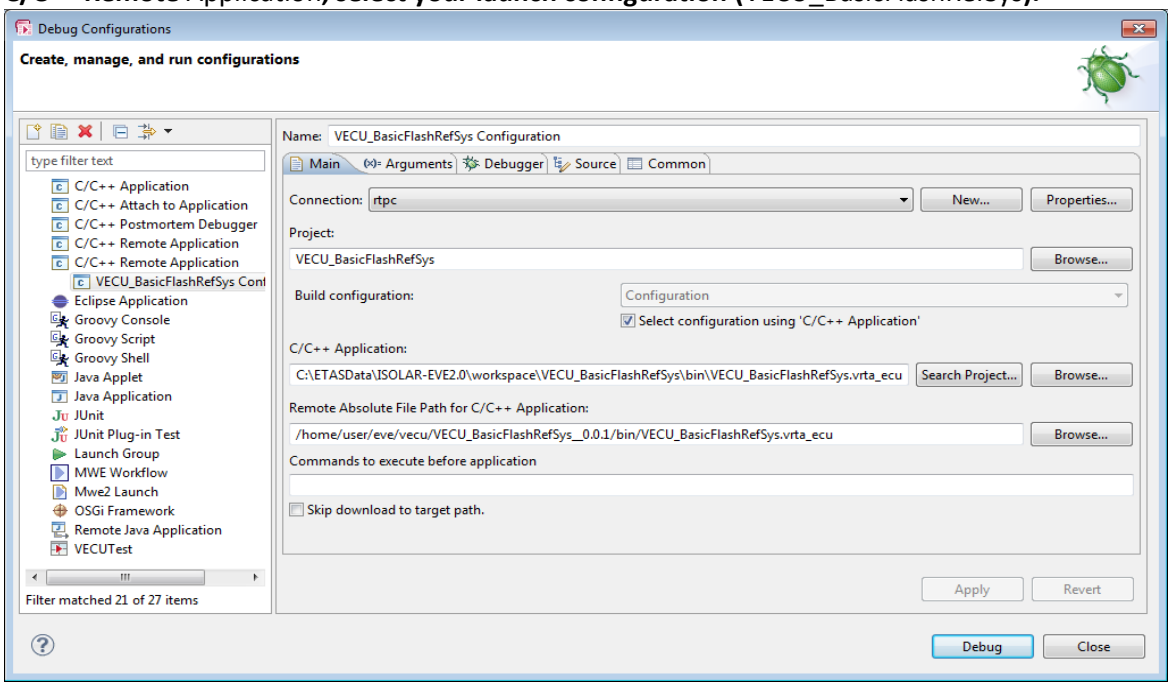
8.1 Setting up a Remote Debugging Configuration

Click **Windows -> Preferences**. Go to **Run/Debug** -> **Launching** and select **Default Launchers.**

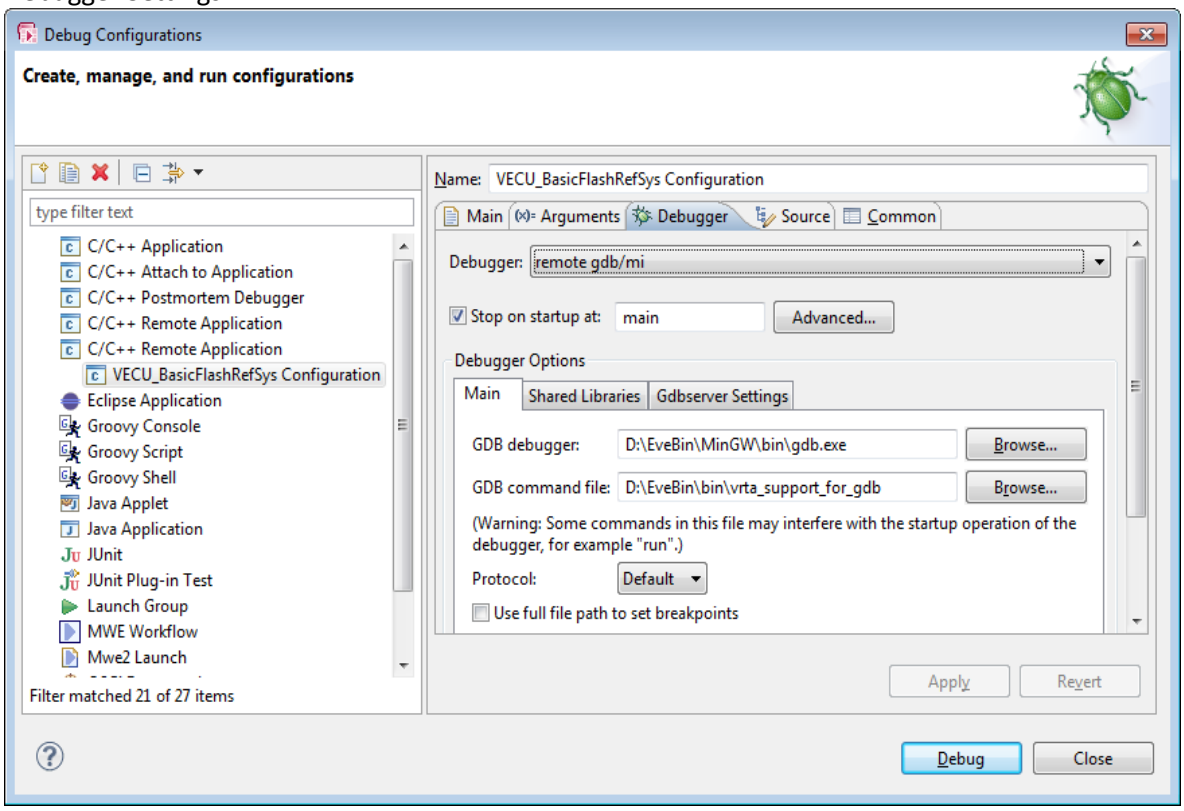
Select Debug under **C/C++ Remote Application** and switch the preferred launcher to **Legacy Remote Create Process Launcher**. Click **Apply** to save the Default Launcher settings.



In the **Project Explorer** window **Right Click** the project and select **DebugAs-> Debug Configurations.** Under **C/C++ Remote** Application**, select your launch configuration (**VECU_BasicFlashRefSys**).**



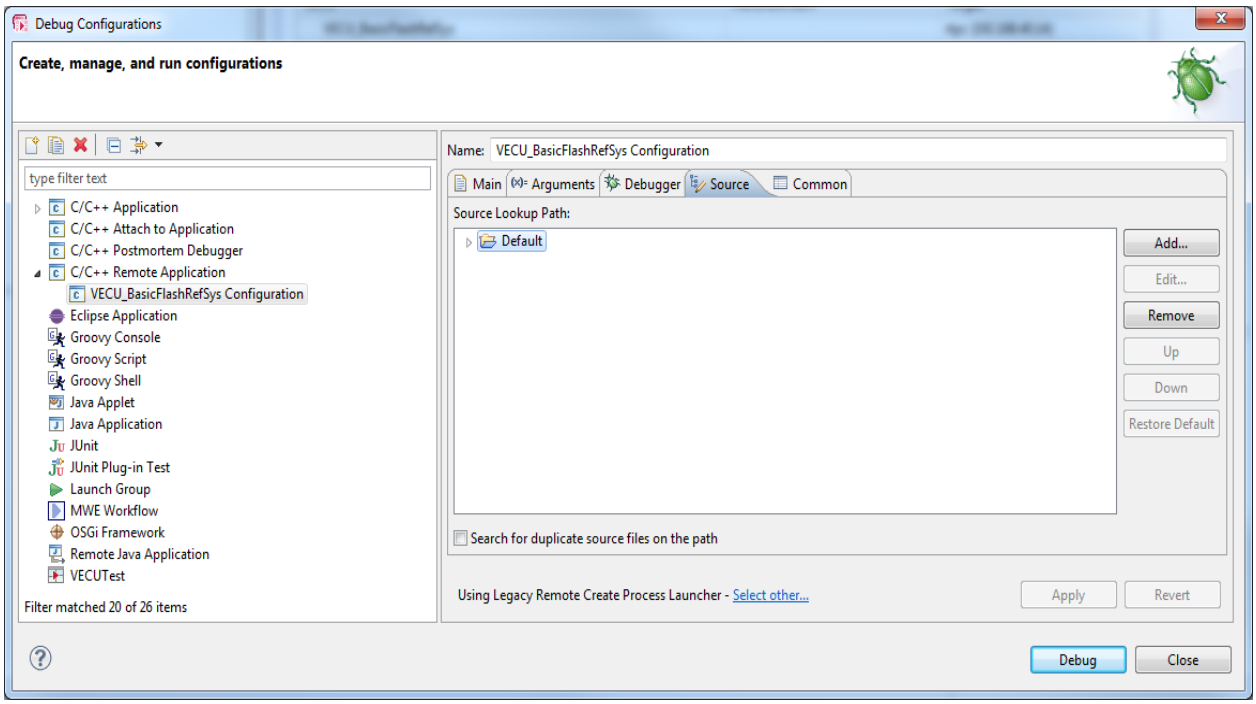
Debugger Settings



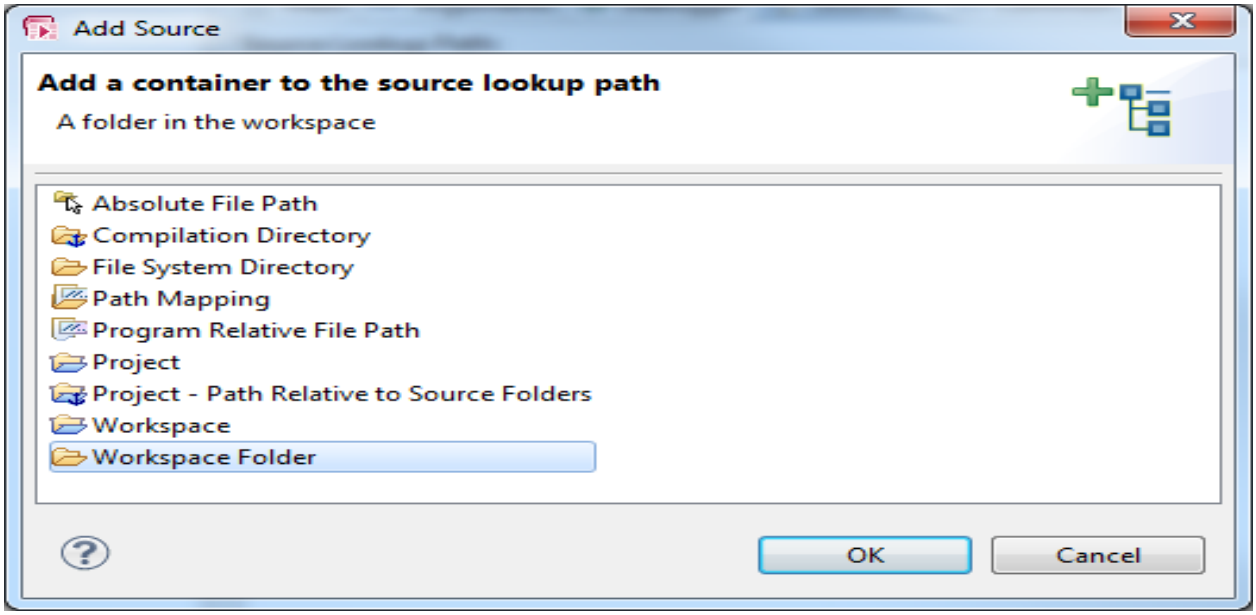
9 Debugging with duplicate source files using Isolar-EVE

If the project has duplicate source files in different folders then the debugger will pick the source file which it finds first. To avoid it picking the wrong version of a source file, you should specify the correct source folders.

In **Source** tab select **Default** and Click **Remove** to remove the default source folder path.

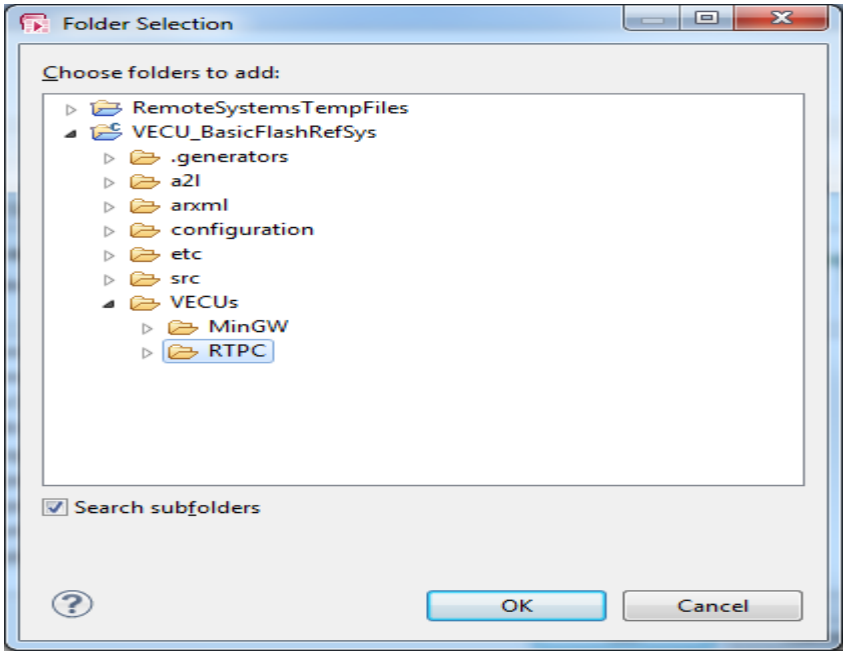


Click **Add**, select **Workspace Folder** and click **ok**



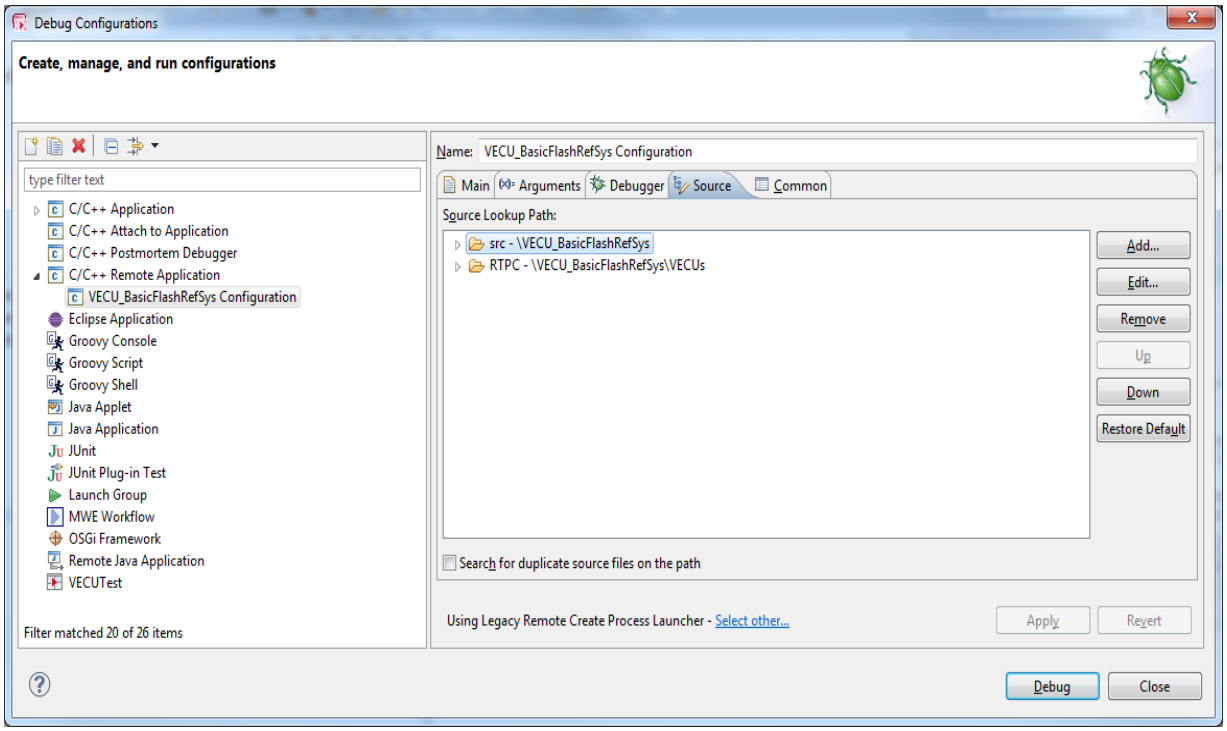
Select the folder which contains the source files that should be used .

Enable 'Search subfolders 'and Click **OK**



You can add multiple folders by repeating this procedure.

Once all the folders are added click **Apply** to save the settings.



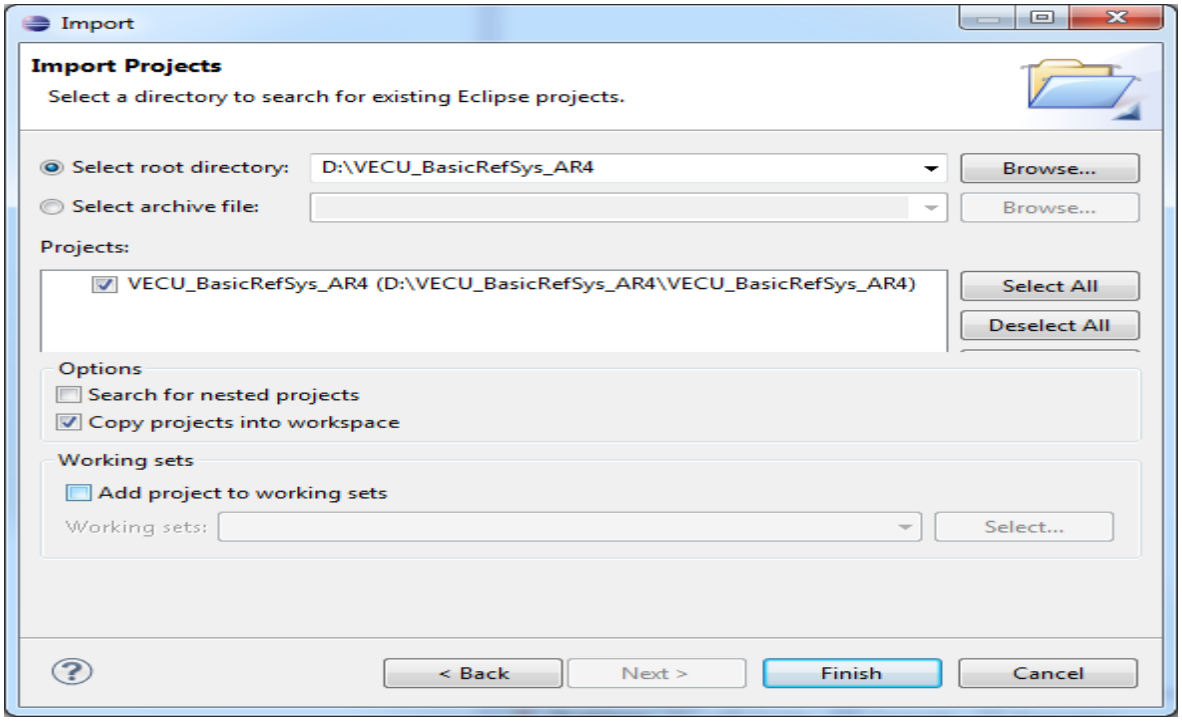
10 Debugging with Eclipse (MinGW)

VRTA-win projects that are built by Isolar-EVE (or Bake for MinGW) can be debugged using Eclipse. Under the hood it uses `gdb` so it is important to ensure that the `vrta_support_for_gdb` file gets loaded.

To debug VRTA ECUs using Eclipse you need the following:

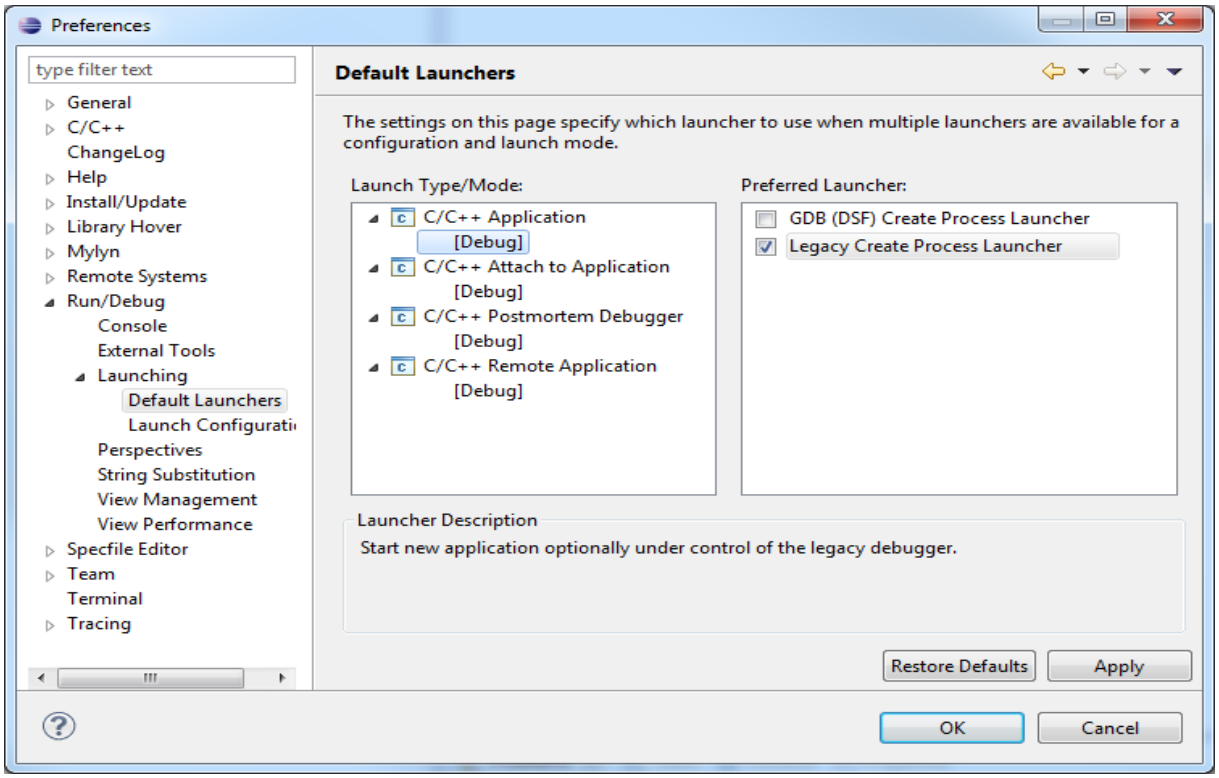
- Eclipse must have the C/C++ support plugins installed. You should install these if necessary.
- The project must have the files `.project`. `.cproject` and `.gdbinit` present.
  Bake will create these if necessary when building a MinGW project.

You then just need to Import the project (File → Import → General → Existing Projects

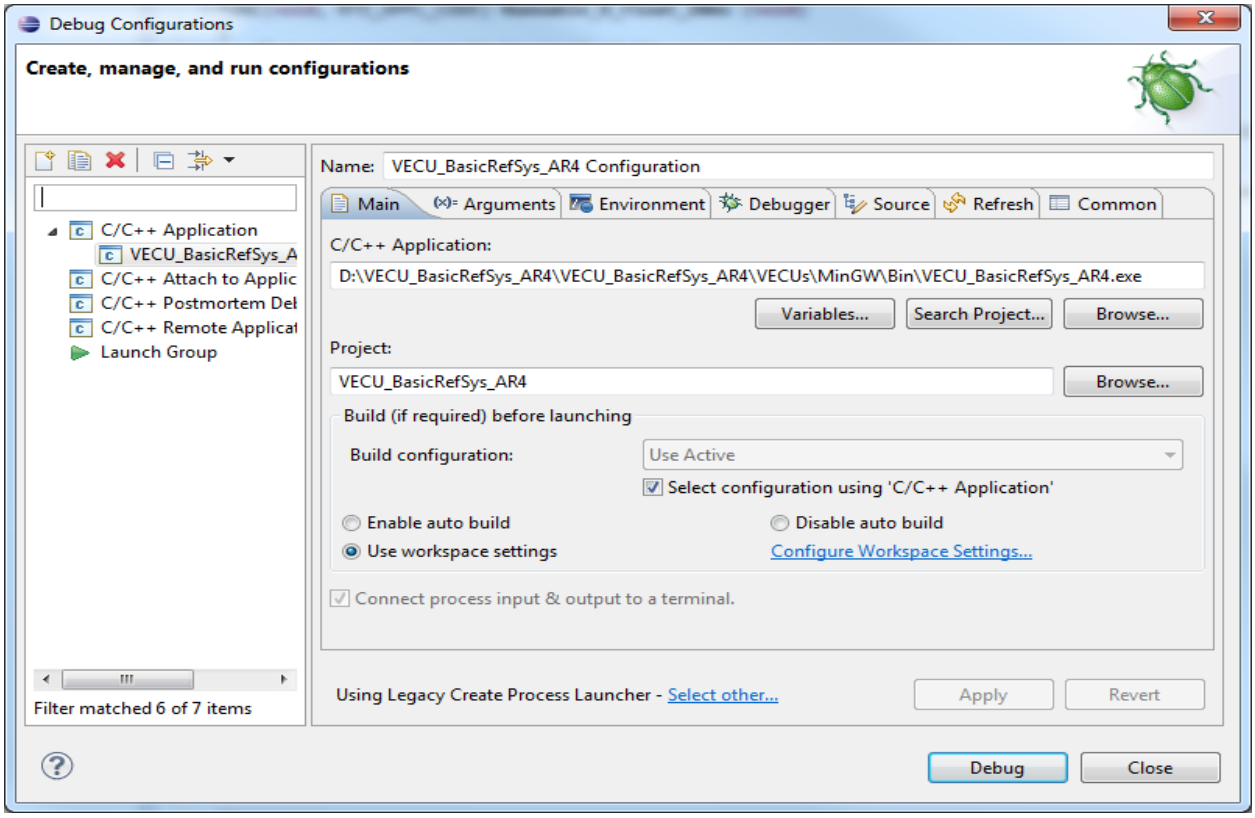


10.1 Debug application

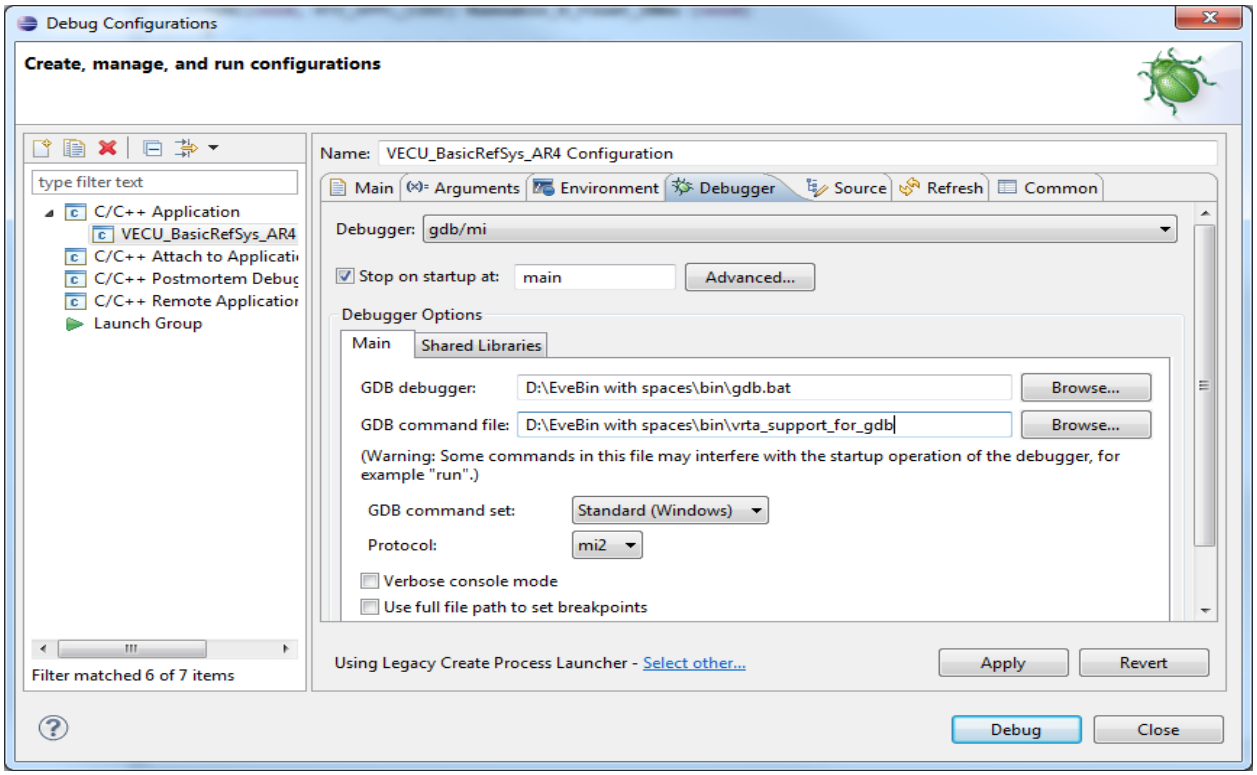
Select **Run -> Debug Configurations.** Under **C/C++ Application**, select your launch configuration



Right click project **Debug As -> Debug Configurations.** Create new **C/C++ Application**

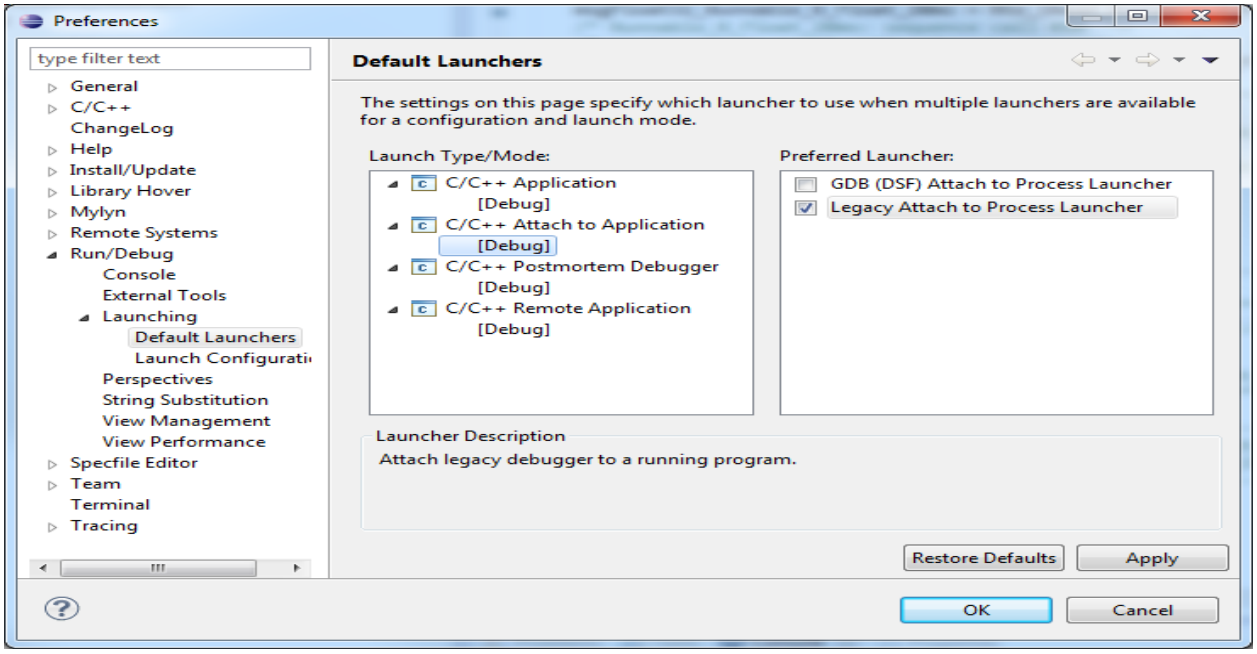


Eclipse must be told where gdb is installed. If this is not on your system PATH, then you need to set it in the Debugger section. You should also set the GDB command file to the `vrta_support_for_gdb` location. Set the GDB command set to Standard (Windows) and protocol to mi2 and debug the application.

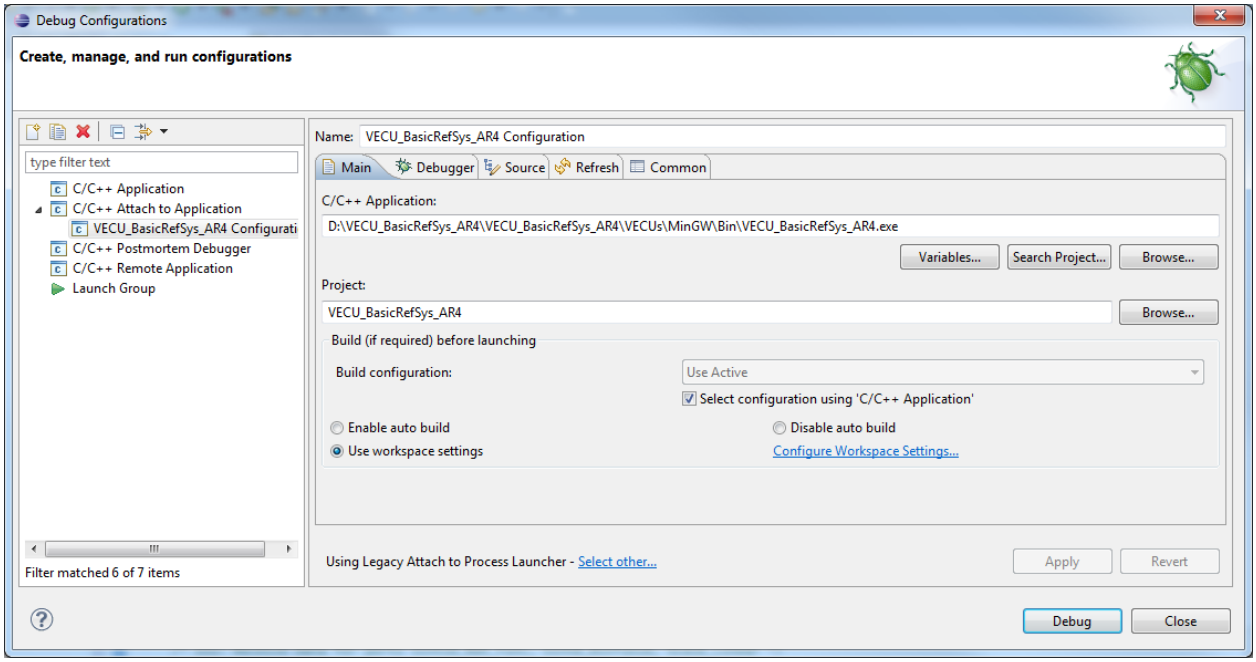


10.2 Debug by Attaching to an application

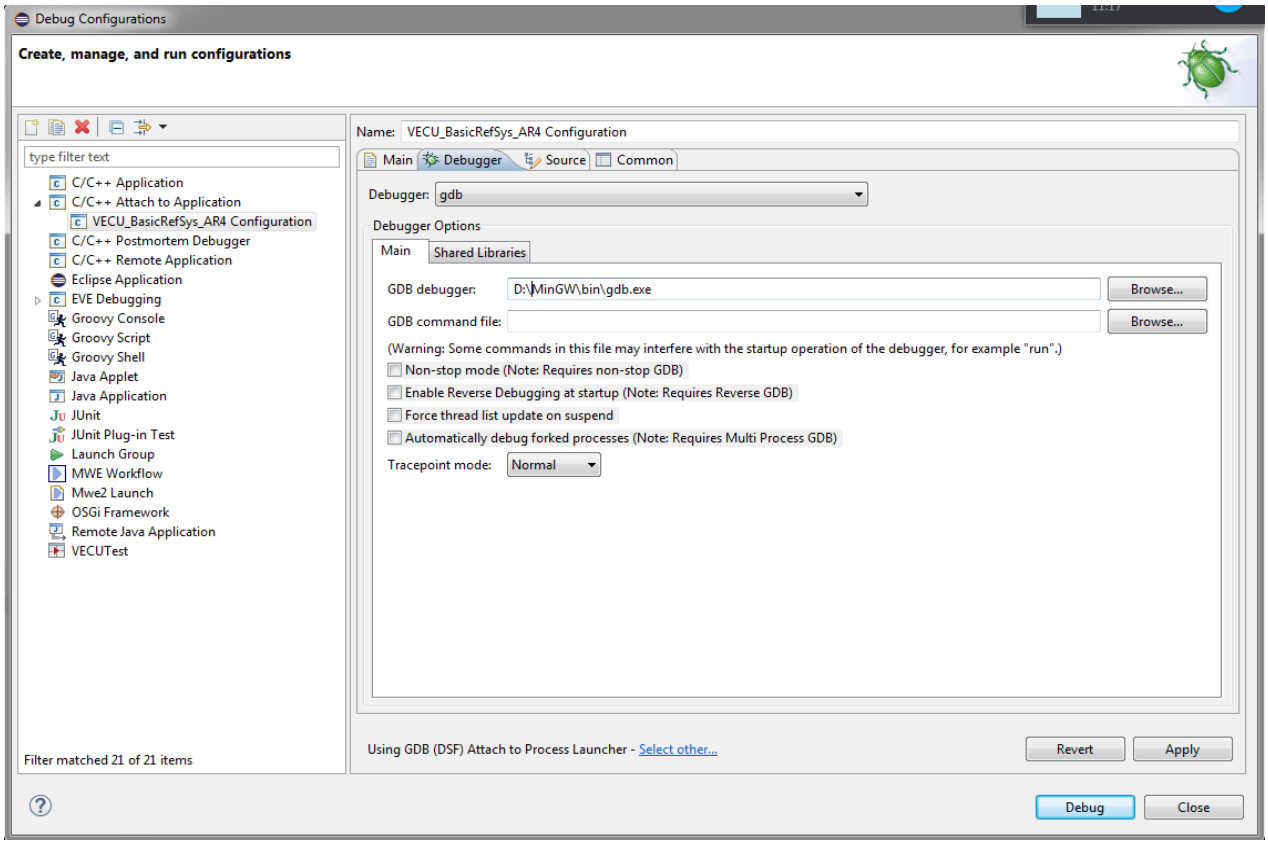
Select **Run -> Debug Configurations.** Under **C/C++ Attach to Application**, select your launch configuration



Right click project **Debug As -> Debug Configurations.** Create new **C/C++ Attach to Application**



Eclipse must be told where gdb is installed. If this is not on your system PATH, then you need to set it in the Debugger section.



11 Remote Debugging with Eclipse (RTPC)

VRTA-ux projects that are built by Isolar-EVE (or Bake for MinGW) can be debugged using Eclipse. Under the hood it uses `gdb` so it is important to ensure that the `vrta_support_for_gdb` file gets loaded.

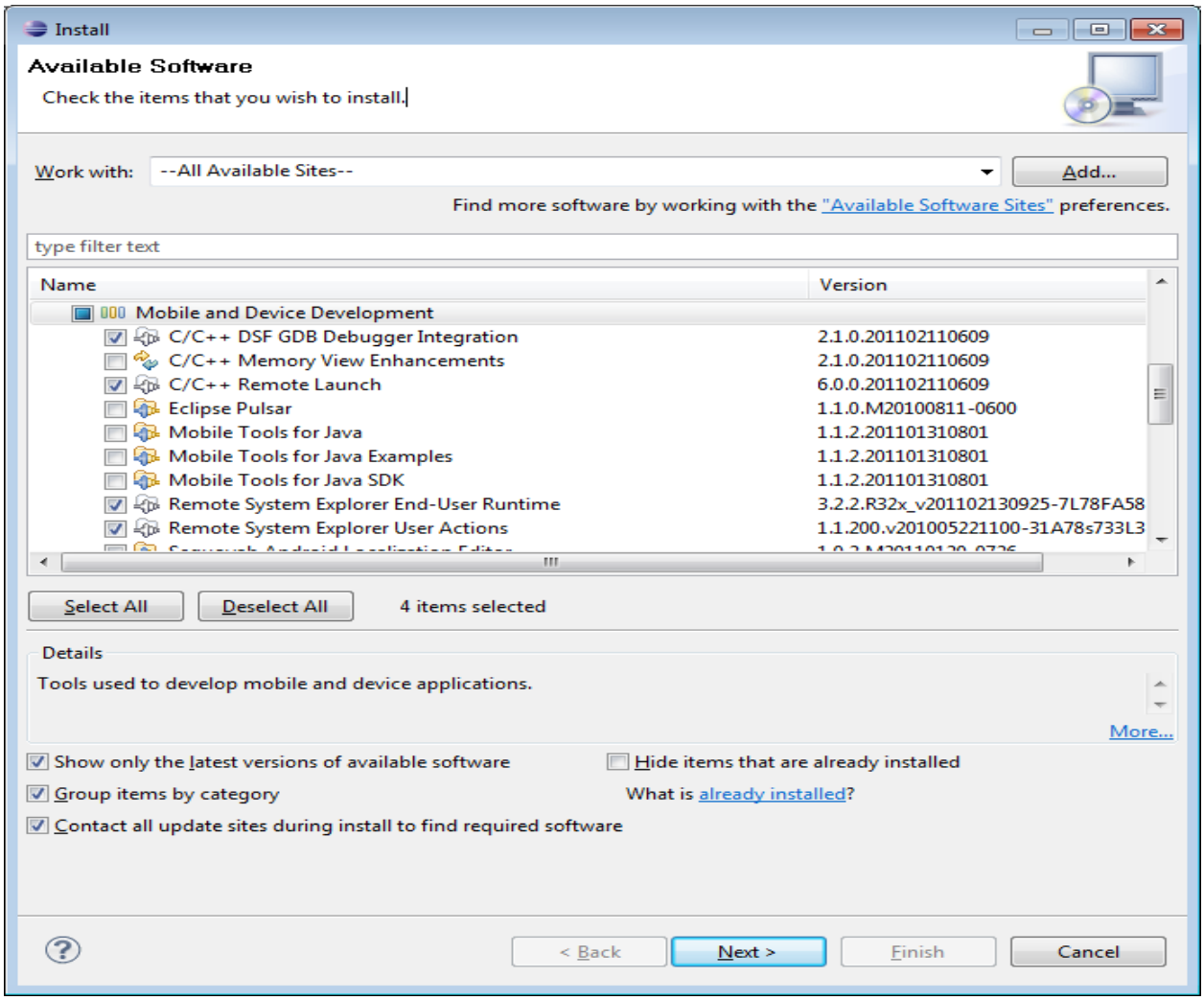
11.1 Prerequisites

You may need to installing the Remote Debugging Plugins before you can perform remote debugging.

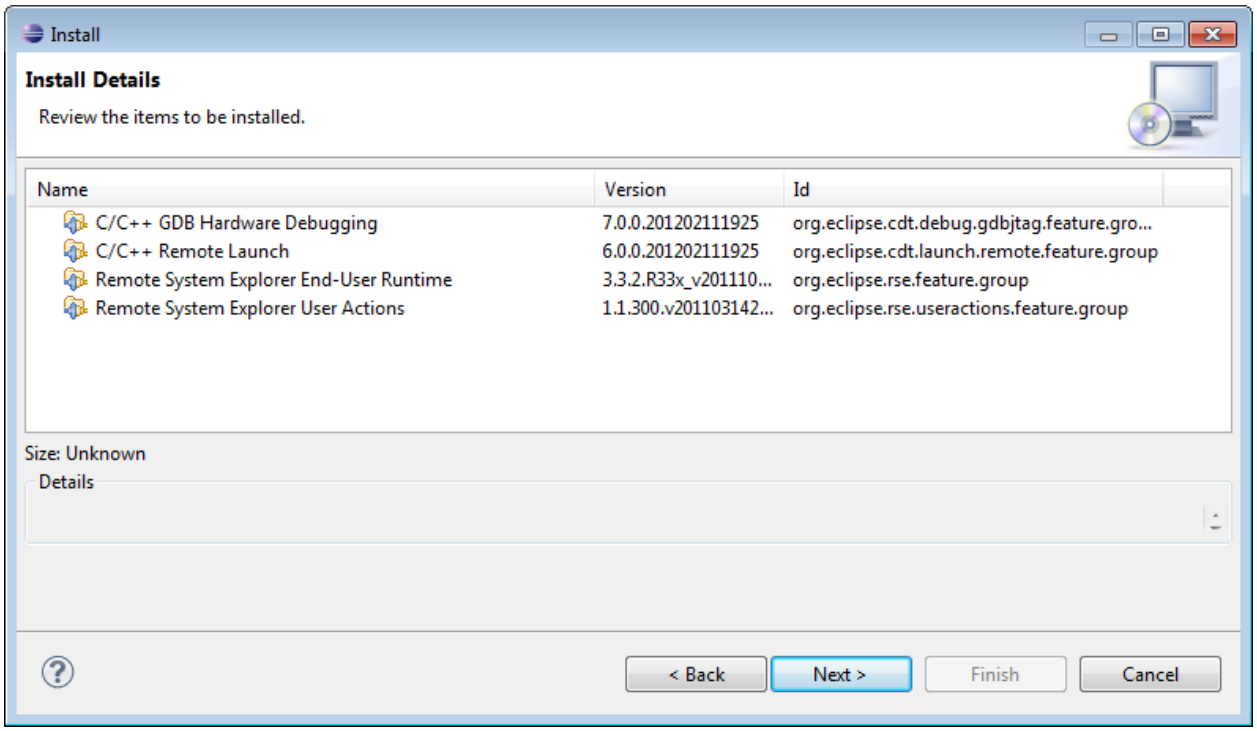
Run Eclipse and select **Help -> Install New Software**. In the **Work with** window, select **All Available Sites**.

Under **Name**, expand **Mobile and Device Development** and select these items:

- C/C++ DSF GDB Debugger Integration
- C/C++ Remote Launch
- Remote System Explorer End-User Runtime
- Remote System Explorer User Actions



Click **Next**, and view the **Install Details**, which tells you if any of the selected items are already present



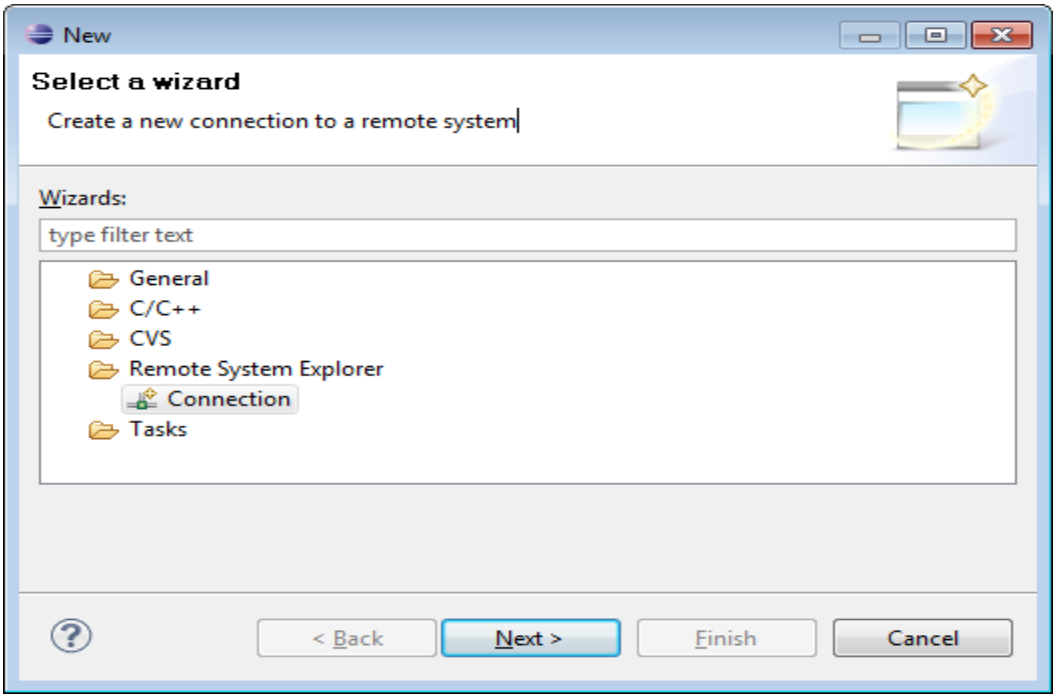
Click **Next**.

Accept the terms of the license agreement and click **Finish**.

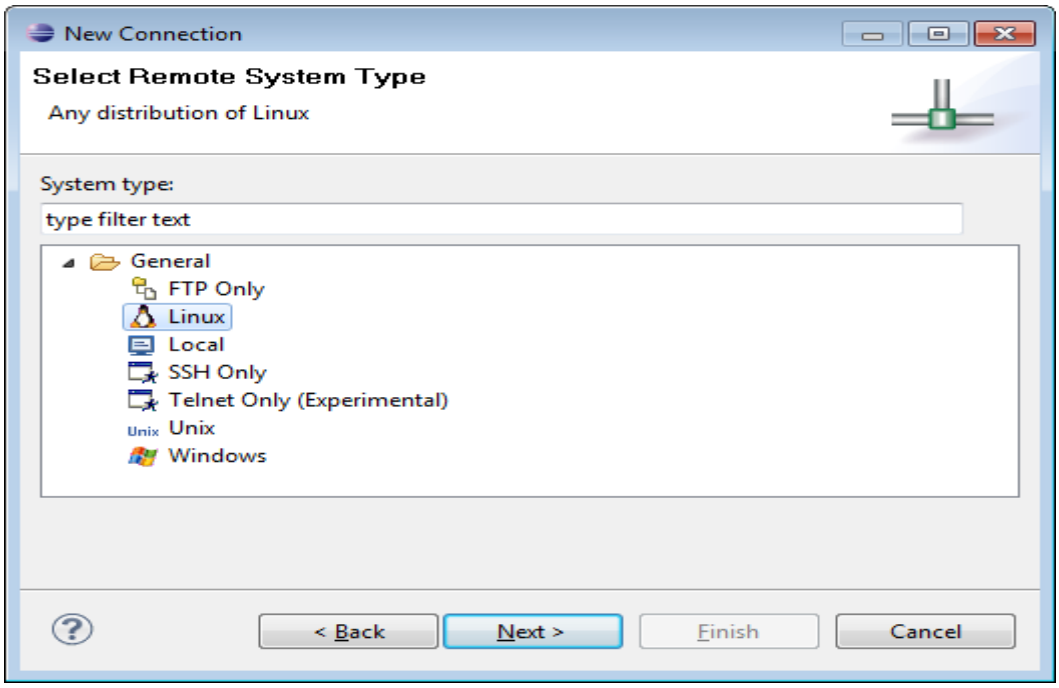
Click **Restart Now**.

11.2 Setting up Eclipse for Remote Debugging

Click **File -> New -> Other**. In the window that appears, double-click **Remote System Explorer** to expand it and select **Connection**.

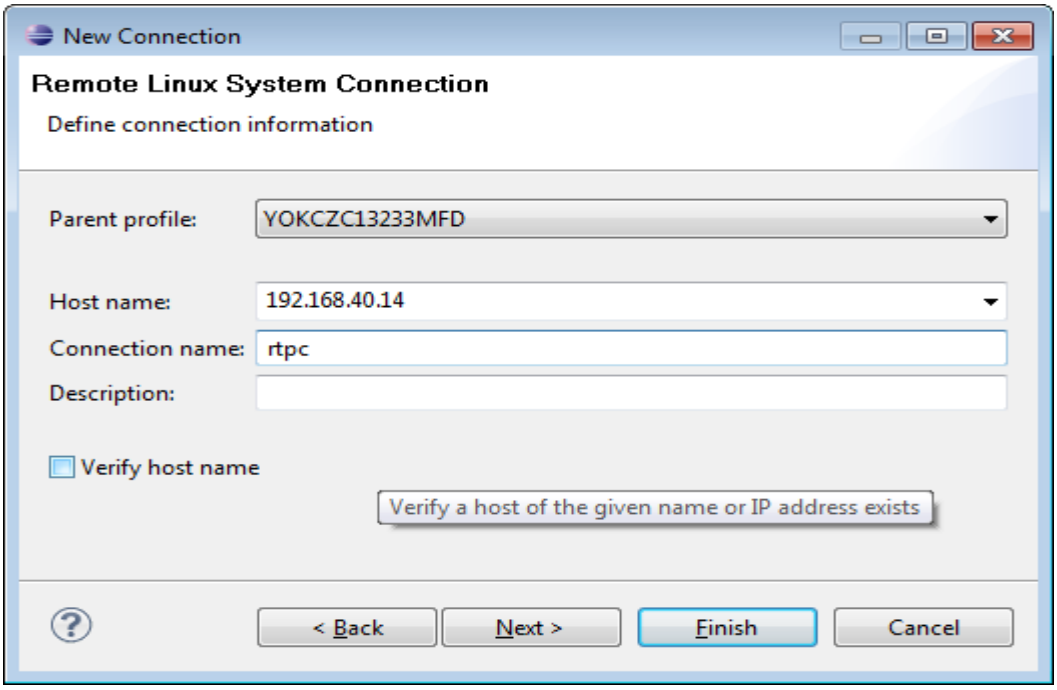


Click **Next**. Select **Linux**

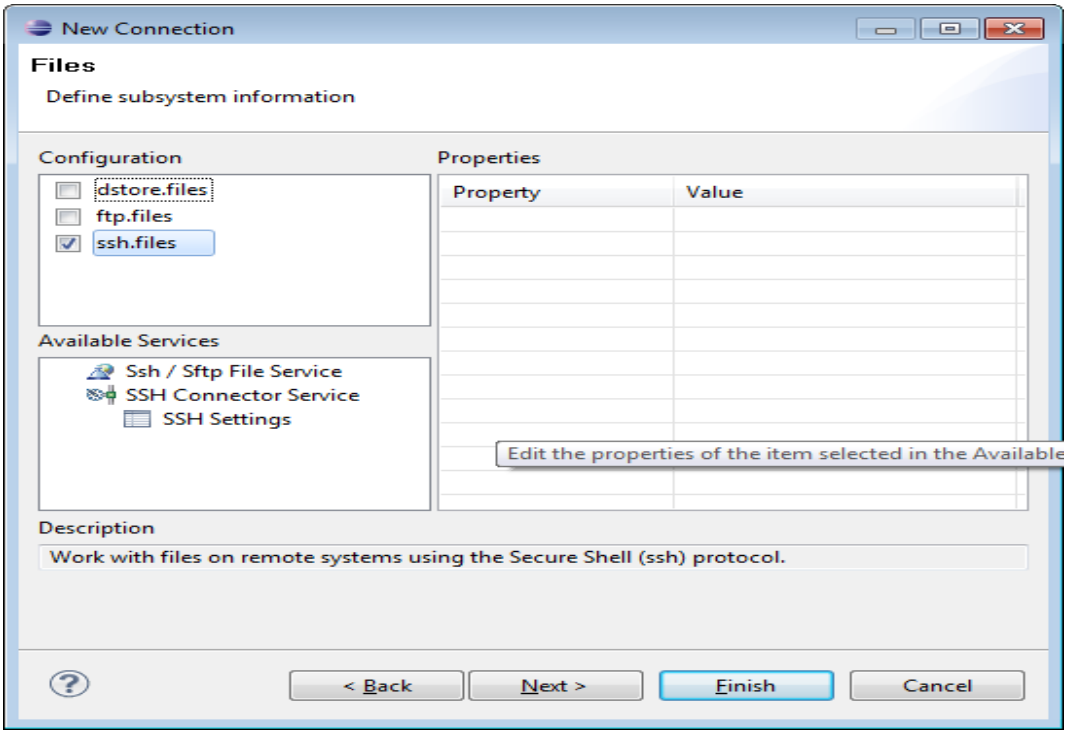


Click **Next**. At **Host name**, enter the IP address of the rtpc (192.168.40.14 in the example).

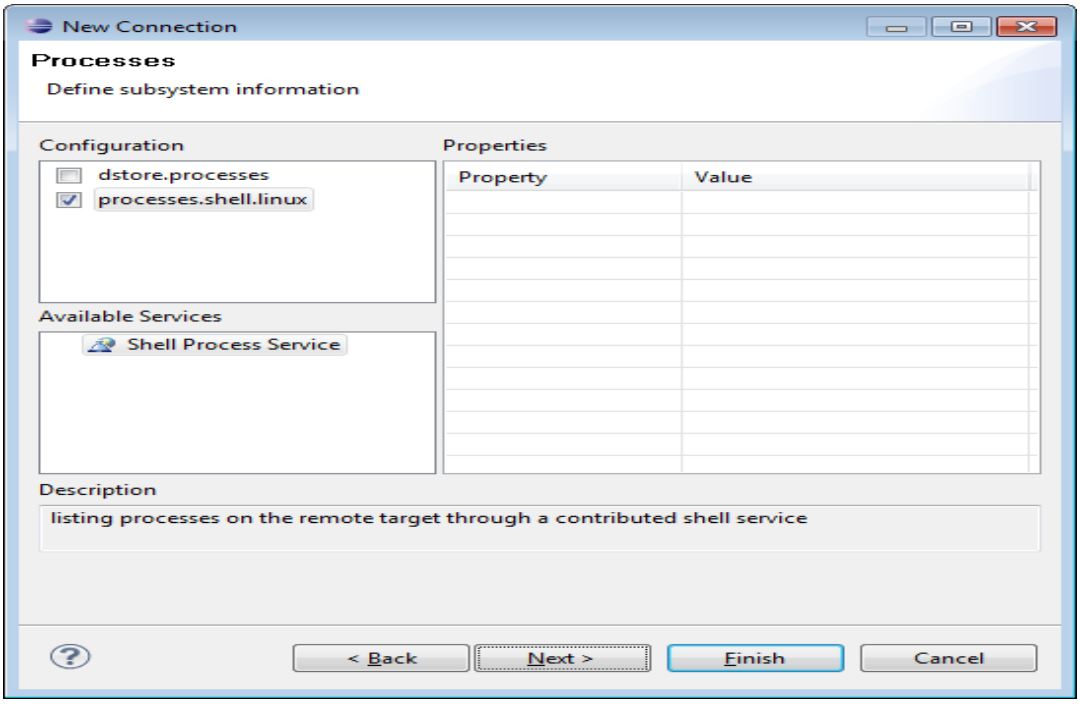
At **Connection name**, enter a name for the connection (rtpc in the example).



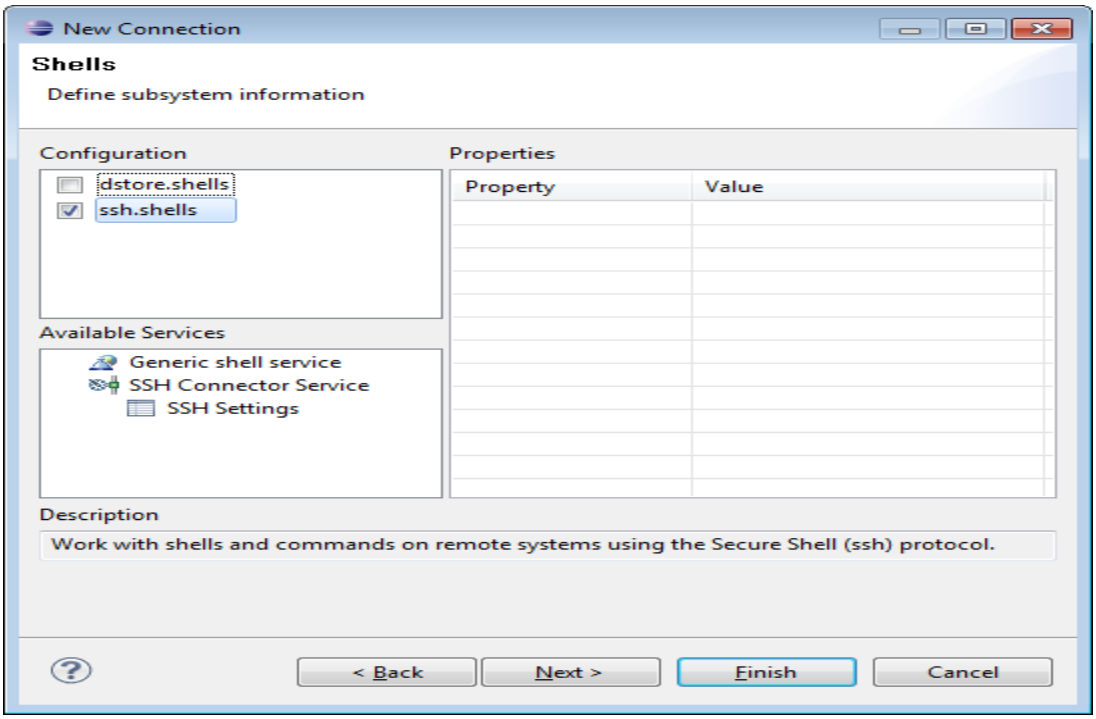
Click **Next**. Under **Configuration**, select **ssh.files**.



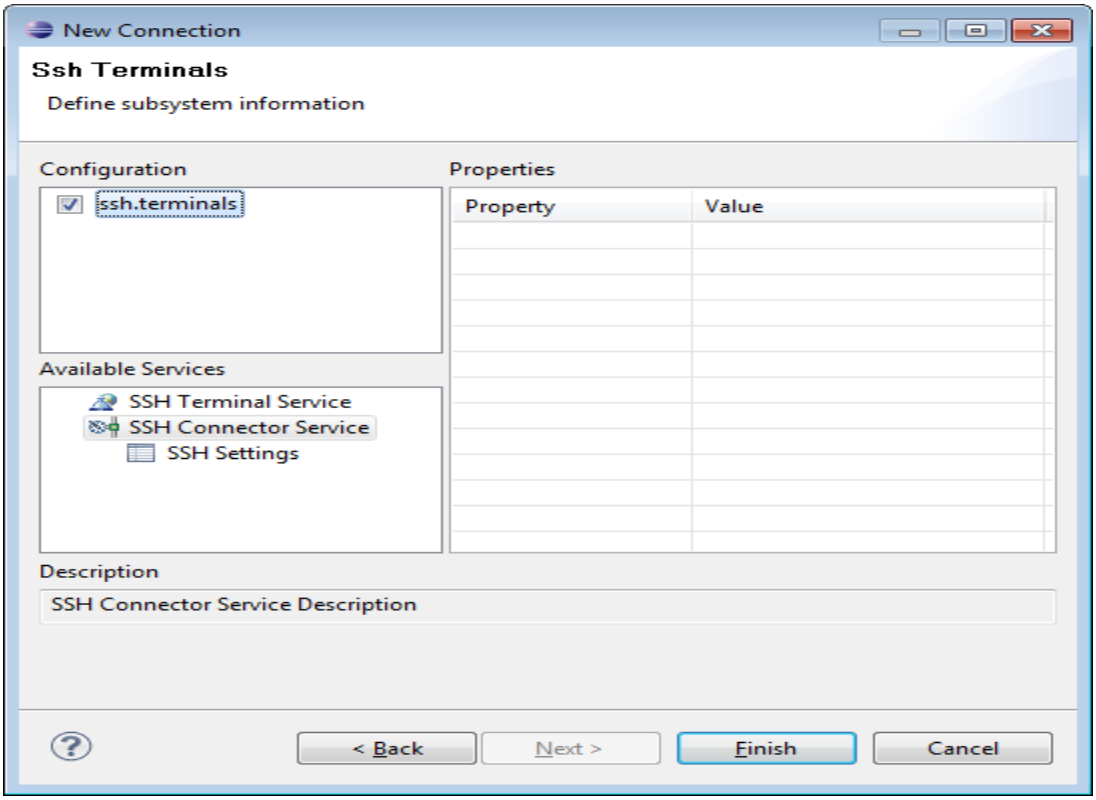
Click **Next**. Select **processes.shell.Linux**



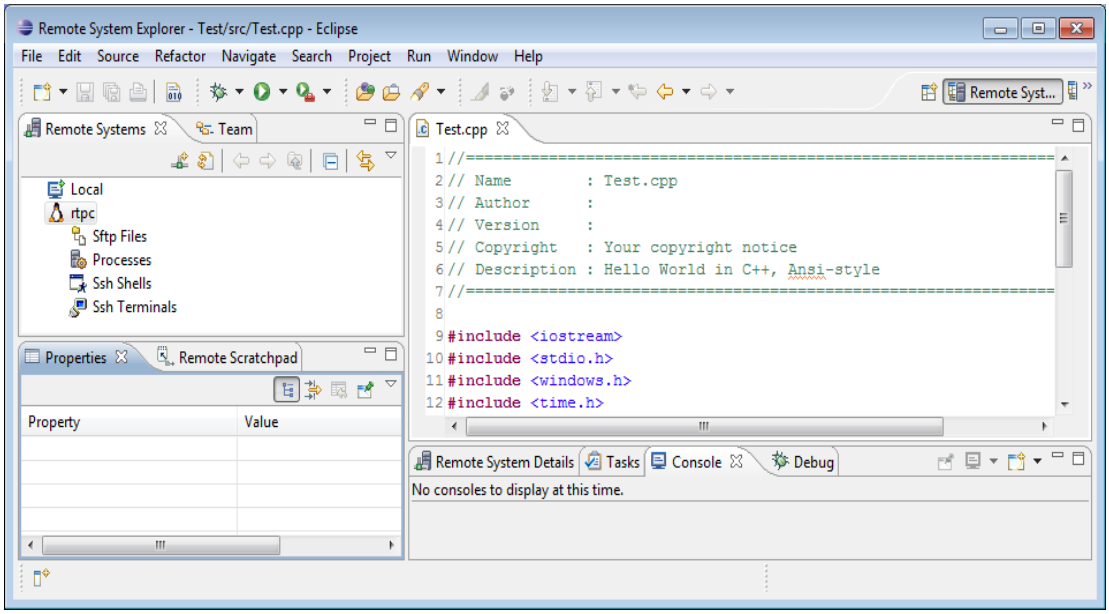
Click **Next**. Select **ssh.shells**.



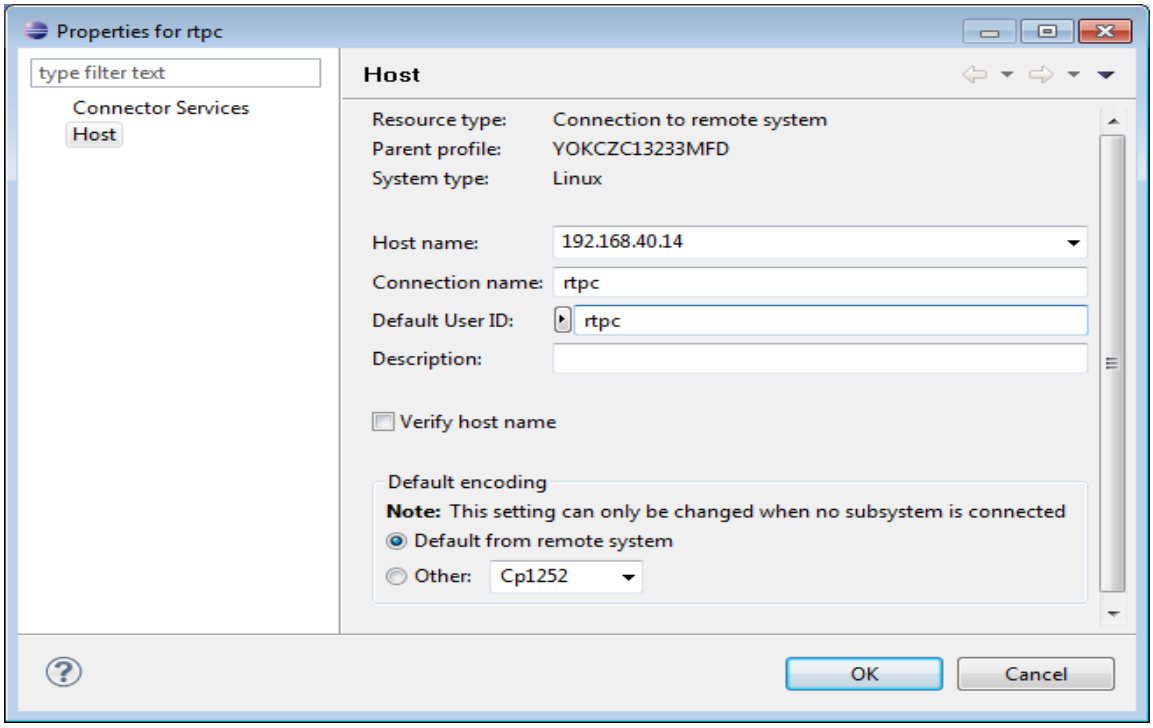
Click **Next**. Select **ssh.terminals**.



Click **Finish**. In the **Remote Systems** tab, right-click the connection and select **Properties**.

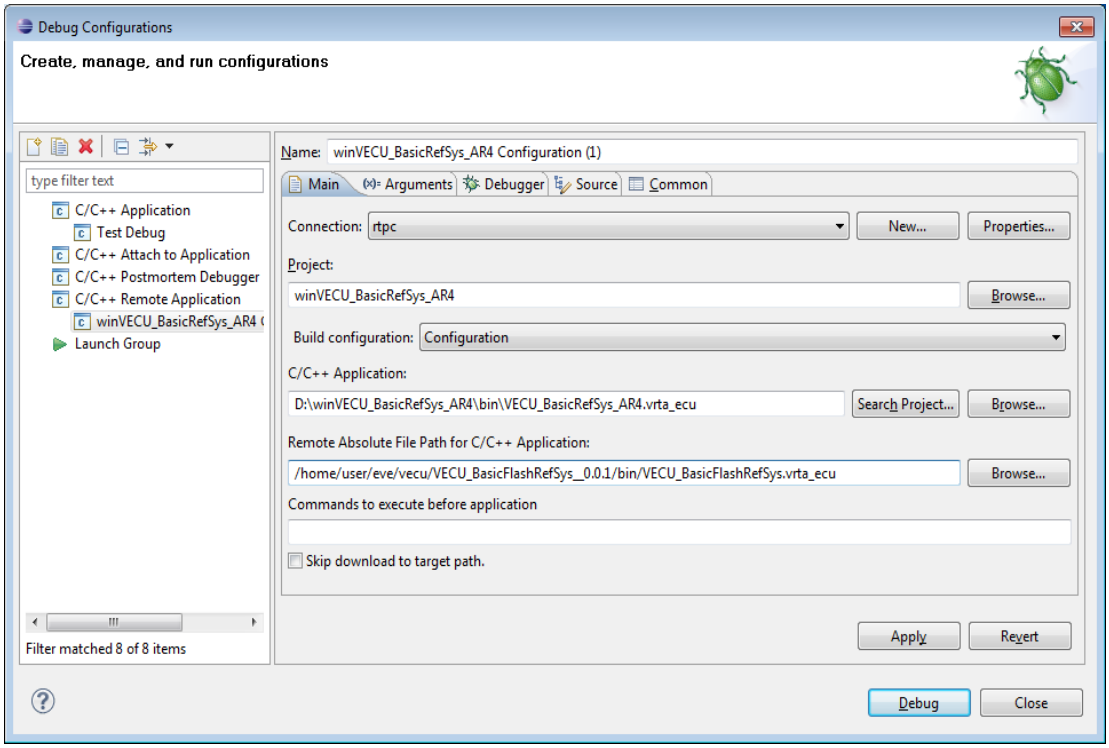


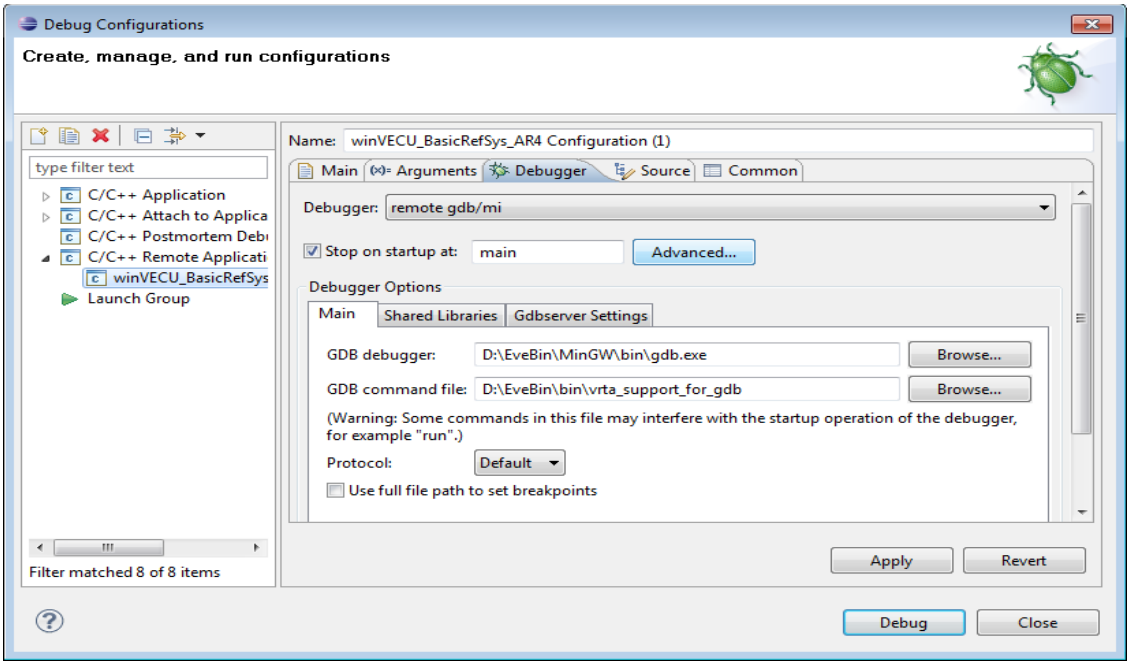
Set the connection's user to **rtpc**: click **Host**. Click the small icon just to the right of **Default User ID** and enter **rtpc and click OK.**



11.3 Setting up a Remote Debugging Configuration

Select **Run -> Debug Configurations.** Under **C/C++ Remote Application**, select your launch configuration **(**VECU_BasicFlashRefSys**).**

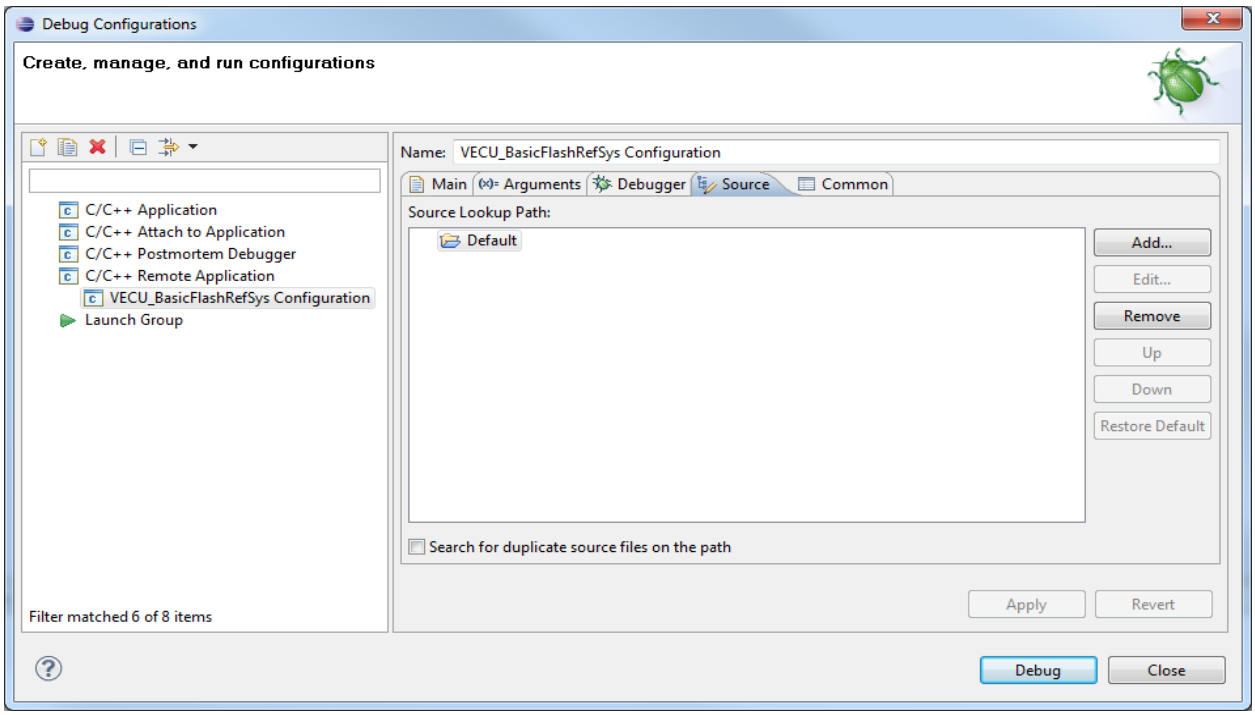




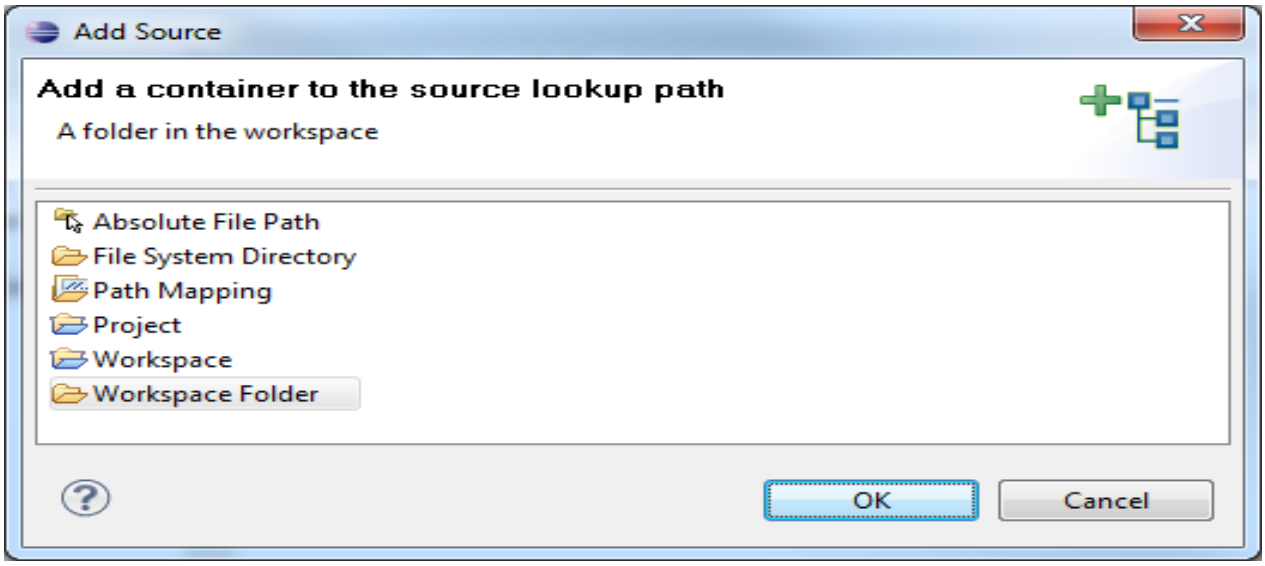
12 Debugging with duplicate source files using Eclipse

If the project has duplicate source files in different folders then the debugger will pick the source file which it finds first. To avoid it picking the wrong version of a source file, you should specify the correct source folders.

In **Source** tab select **Default** and Click **Remove** to remove the default source folder path.

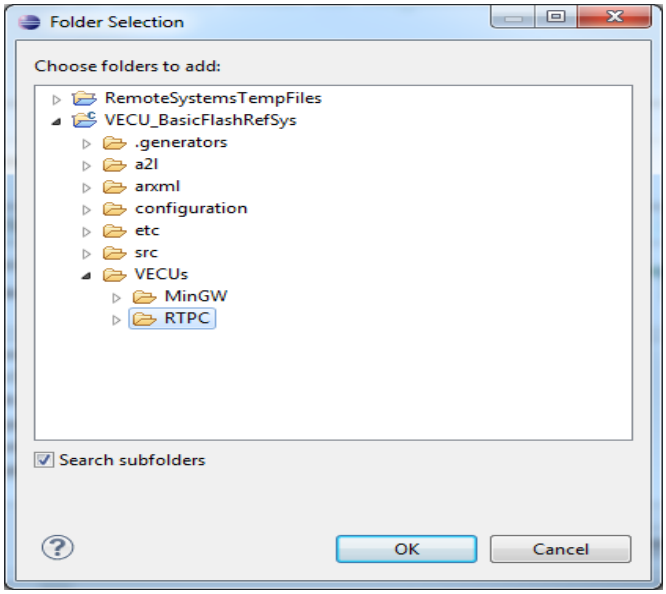


Click **Add**, select **Workspace Folder** and click **ok**



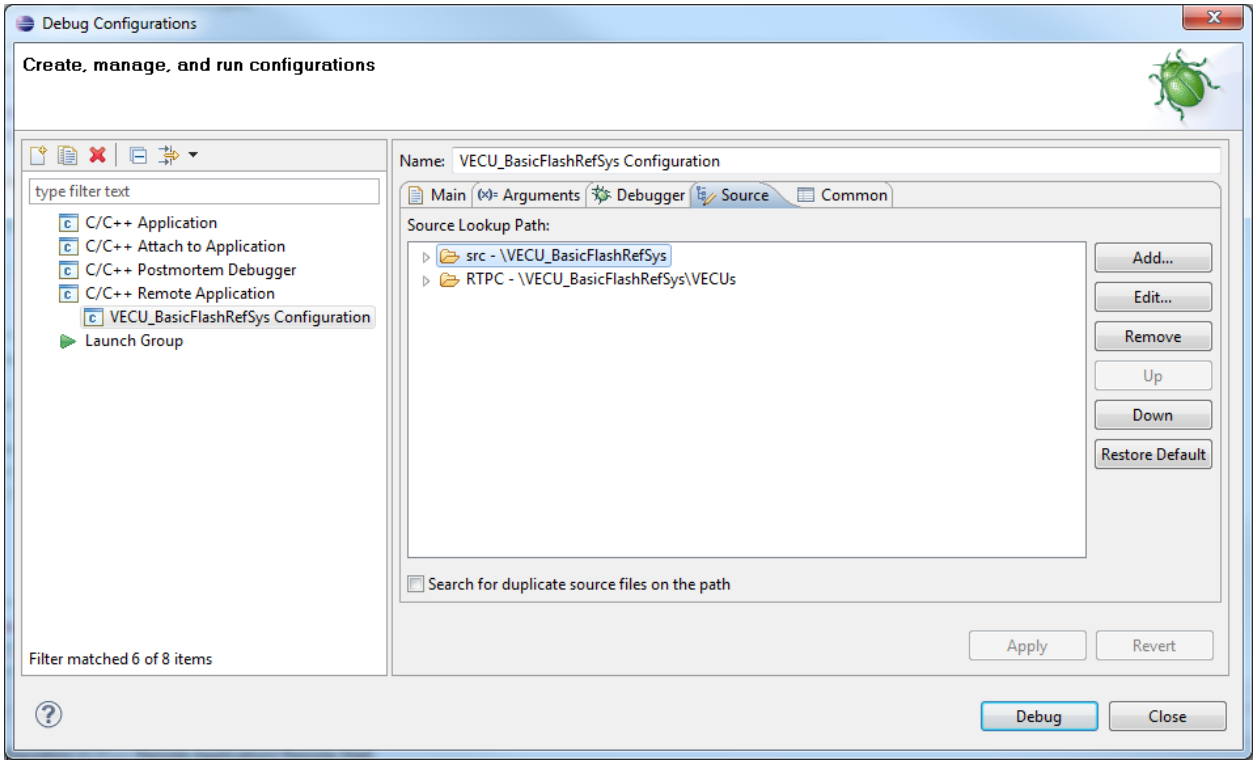
Select the folder which contains the source files that should be used .

Enable 'Search subfolders 'and Click **OK**



You can add multiple folders by repeating this procedure.

Once all the folders are added Click **Apply** to save the settings.

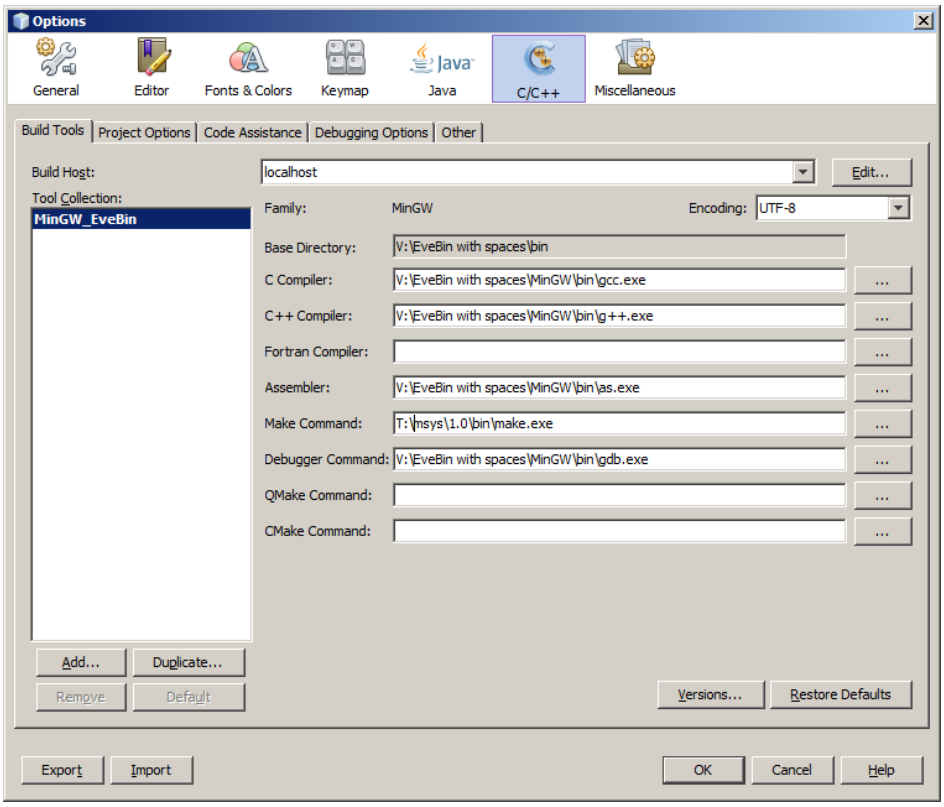


13 Debugging with NetBeans (MinGW)

(Tested with NetBeans IDE 7.3 with C++ plugins)

NetBeans can be used to debug VRTA executables built for MinGW. It uses the standard `gdb` debugger.

First ensure the NetBeans knows where your MinGW installation exists (Tools → Options → C/C++)



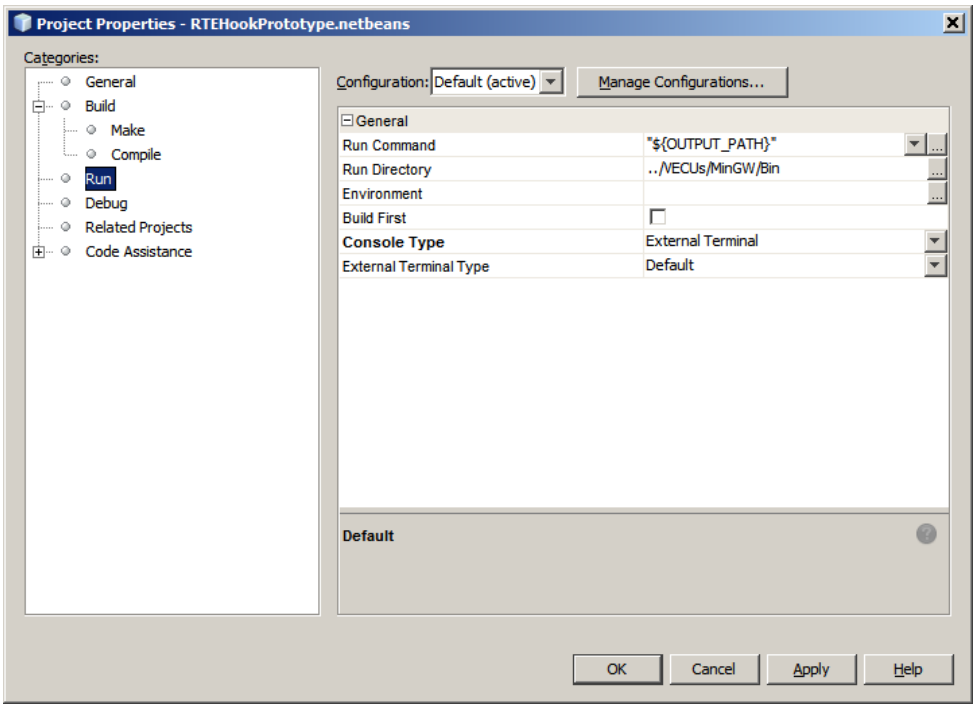
Then add a line to your default `.gdbinit` file of the form "`source <path to vrta target>\vrta_support_for_gdb`". The likely location for your `.gdbinit` file is `C:\Users\<username>\.gdbinit` (see below). You might have to create the file if it does not already exist.

e.g. `source C:\ETAS\RTA-OS\Targets\VRTA_1.0.0\vrta_support_for_gdb`

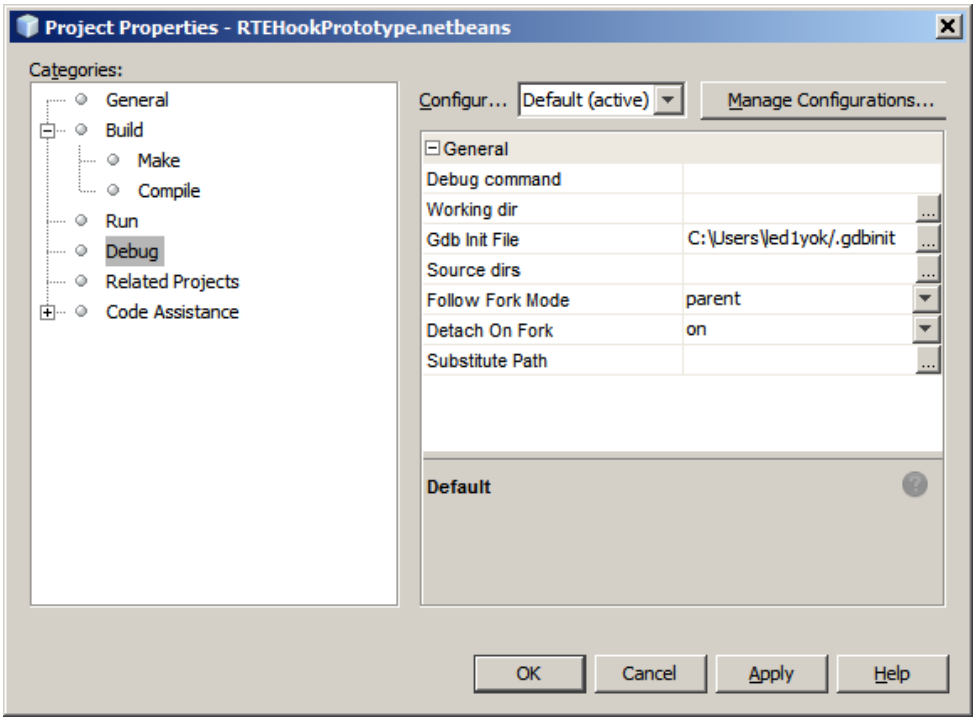
Adding a VRTA executable to NetBeans is simple.

- Select File → New Project → C/C++ Project from Binary File
- Select the .exe file (e.g. D:\Working\DevTrunk\Common\Bake\SampleApps\RTEHookPrototype\VECUs\MinGW\Bin\RTEHookPrototype.exe)
- Add the Sources root (if NetBeans doesn't already find it)
- Save the project

- Go to the Project Properties and set the Run Console Type to **External Terminal** .



- Check the location of the `.gdbinit` file in the Debug section.

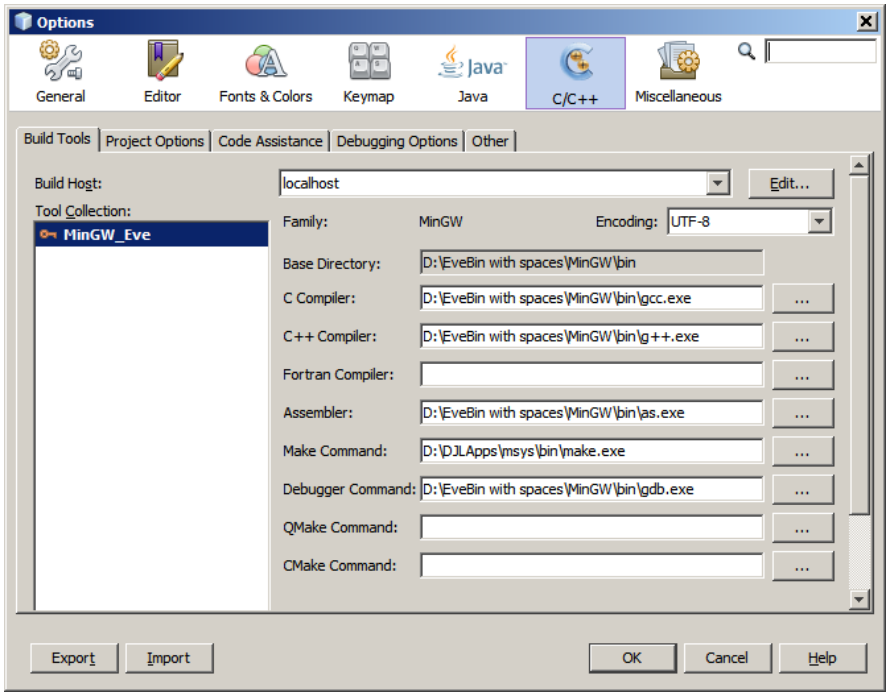


- Go to the 'Source Files' node in the project manager and open up one of the files that you want to debug.
- Set any breakpoints and start debugging! (Ctrl-F5)

14 NetBeans additional – Building and debugging Eve projects

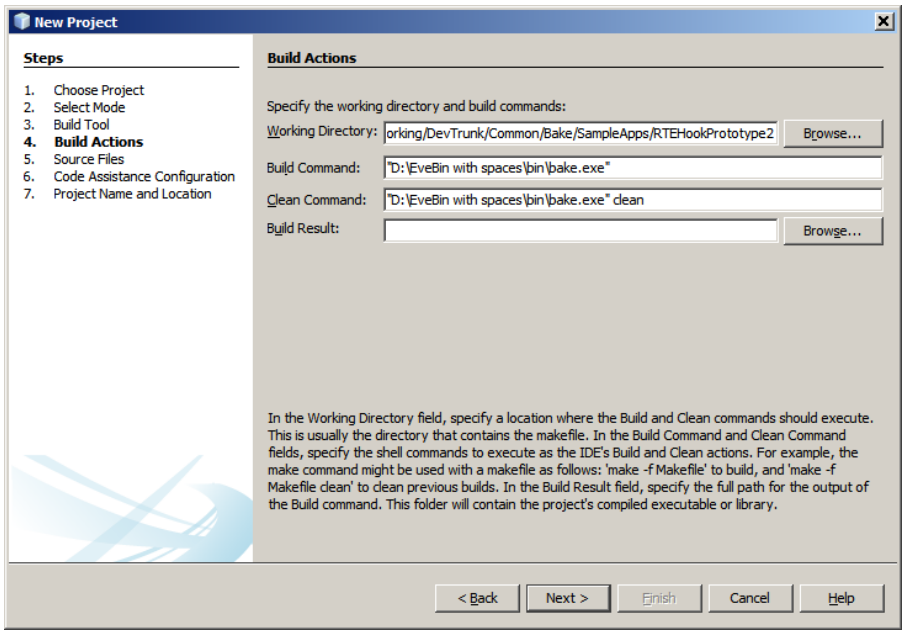
You can also use NetBeans to edit and build Bake-based projects.

You will need to have MySys installed on your PC in addition to MinGW (https://netbeans.org/community/releases/69/cpp-setup-instructions.html#compilers_windows) because the Build Tools requires that `make.exe` is available (though we don't actually use it).



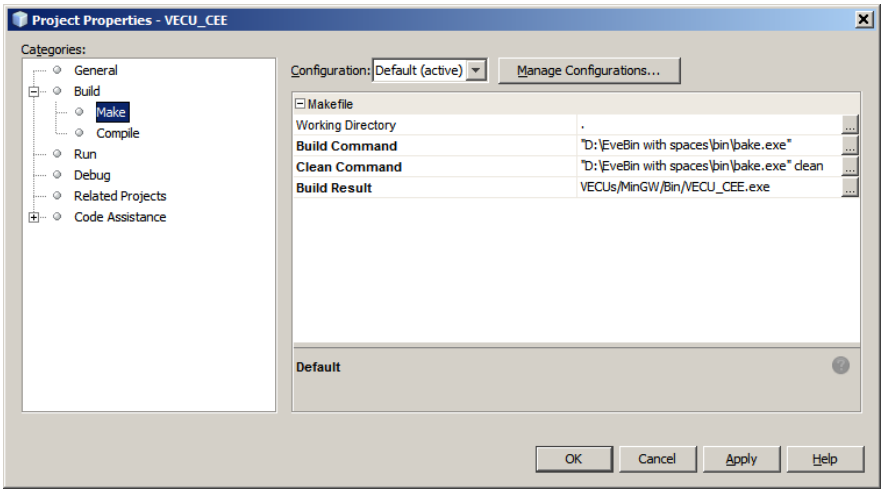
The steps are:

- Create a dummy `Makefile` in your project root (content e.g. `# Dummy makefile`)
- Select File → New Project →
  - **Choose Project:** C/C++ Project with Existing Sources
  - **Select Mode:** Specify the project root folder (e.g. `D:\Working\DevTrunk\Common\Bake\SampleApps\VECU_CEE`) and **Custom** configuration mode.
  - **Build Tool:** Leave as default
  - **Build Actions:** Change Build Command and Clean Command:



- **Source files:** Add the folders that contain your sources
- **Code assistance:** No change
- **Project Name/location**: As desired

- When you press Finish, the project will be rebuilt using Bake.
  (Might not work first time? If not, select Clean and Build)
- Go back to the project properties and add the .exe to the Make Build Result.



- Also set the Run Console Type to **External Terminal** .
- This will now allow you to run the executable within NetBeans.

You can debug as described previously.